

实验报告

姓名：时分

学号：PB23151799

稀疏一元多项式运算器 实验报告

设计目标

在C语言环境下，设计一个稀疏一元多项式运算器，要求实现如下功能：

1. 输入并创建多项式（按指数升序或降序排列，系数浮点型，指数整型）

输入浮点型系数和整型指数，建立多项式并存储。

2. 输出多项式，项数+每项系数指数（按指数升序或降序排列）

3. 求和：多项式相加

输入加数多项式的序号，输出相加结果并询问是否储存。

4. 求差：多项式相减

输入被减数与减数多项式的序号，输出相减结果并询问是否储存。

5. 求值

输入多项式序号与浮点型数值，输出多项式在该数处的取值

6. 销毁

输入多项式序号，删除多项式全部内容

7. 清空

输入多项式序号，将多项式清零

8. 修改

1. 插入新项：输入多项式序号、新项的系数与指数，将新项插入多项式

2. 删除项：输入多项式序号、需删除项的指数，将该项从多项式中删除

3. 修改已有项：输入多项式序号、需修改项的指数、修改后的系数与指数，修改该项

9. 微分（N阶导数）

输入多项式序号和需求的微分阶数，输出微分结果并询问是否储存

10. 不定积分

输入多项式序号，输出不定积分结果并询问是否储存

11. 定积分

输入多项式序号与定积分起点、终点，输出定积分结果

12. 乘法和乘方

1. 乘法：输入乘数多项式的序号，输出相乘结果并询问是否储存

2. 乘方：输入乘方多项式的序号与乘方次数，输出乘方结果并询问是否储存

13. 带余除法

输入被除数与除数多项式的序号，输出商多项式与余数多项式，分别询问是否储存

算法设计

数据结构

采取菜单式驱动，循环输出菜单界面，输入操作序号，执行对应操作。

采用带头结点的链表存储多项式，链表类型定义如下：

```
typedef struct LNode{
    double coef; //该项系数
    int exp;      //该项指数
    struct LNode *next;
}LNode,*LinkList;
```

使用链表头结点的exp变量存储链表长度。

采用链表指针数组 p[NUM] 存储各链表的头指针，数组元素初值为NULL，创建链表时从头开始查找空位置，将链表头指针存入第一个空位置中。

除法功能结果较为特殊，包含商多项式和余数多项式，故采用两个链表头结点组成的结构体类型存储除法的结果，类型定义如下：

```
typedef struct DivResult{
    LinkList quotient; //商多项式的头指针
    LinkList remainder; //余数多项式的头指针
}DivResult;
```

程序结构

各函数原型、功能和接口的描述如下：

```
int Locate(LinkList L,int nexp);
    //查找多项式内是否有指定的指数，有则返回该项的位置，没有返回0
Status InitList(LinkList &L);
    //在指定位置创建系数从高到低的多项式，包括输入功能，输入-1结束输入
Status InitList(LinkList L);
    //输出指定多项式
LinkList AddList(LinkList La,LinkList Lb);
    //创建一个新多项式，内容为两个输入的多项式相加，返回结果的头指针
LinkList SubList(LinkList La,LinkList Lb);
    //创建一个新多项式，内容为两个输入的多项式相减，返回结果的头指针
double EvaluateList(LinkList L,double x);
    //代入x的值计算多项式的值并返回
Status DestroyList(LinkList &L);
    //释放链表L包括头结点内的所有结点
Status ClearList(LinkList &L);
    //释放链表L头结点外的所有结点
Status InsertNode(LinkList &L,int nexp,double ncoe);
    //在链表L中按顺序插入新结点
Status DeleteNode(LinkList &L,int location);
    //删除链表L指定位置的结点
Status EditNode(LinkList &L,int location,int nexp,double ncoe);
    //修改链表L指定位置的结点
LinkList DiffList(LinkList L);
    //计算L的一阶导数存入新链表，返回新链表的头指针
```

```
LinkedList IntelList(LinkedList L);  
    //计算L的不定积分（常数项为0）存入新链表，返回新链表的头指针  
LinkedList MultList(LinkedList La,LinkedList Lb);  
    //创建一个新多项式，内容为两个输入的多项式相乘，返回结果的头指针  
DivResult DivList(LinkedList La,LinkedList Lb);  
    //输入被除式与除式，以结构体形式返回商和余数的头指针  
int menu(void);  
    //打印菜单界面，输入操作序号并返回
```

调试分析

各功能测试数据与问题解决如下：

1. 输入并创建多项式：

1. 常规输入 1 2.5 3 6.1 8 -2.3 -1，没有出现问题。
2. 输入负指数 -2 1 -1，问题：创建了含负指数结点。
解决方法：在输入指数时添加合法性检测，只允许输入非负整数和-1（结束指令）。
为插入、修改同样加入该检测。
3. 输入重复指数 1 1 1 1 -1，问题：创建了指数重复的结点。
解决方法：在输入指数后利用Locate函数检测是否重复，若重复则提示重新输入。
为插入、修改同样加入该检测。
4. 输入零系数 2 0 -1，问题：创建了零系数结点，空占位置。
解决方法：在输入系数时检测是否为0，若为0则不创建结点。
为插入、修改同样加入该检测。

2. 输出多项式：

1. 常规输入，没有出现问题。
2. 输入未创建的序号，问题：无法输出未创建的多项式。
解决方法：为各功能输入序号时添加合法性检测，若当前位置元素为NULL，则提示未创建多项式。
3. 输入仅含头结点多项式的序号，问题：无法输出仅含头结点的多项式。
解决方法：在输出前增加判断，若仅含头结点，则输出0.000。

3. 求和：

1. 常规输入，没有出现问题。
2. 输入含有指数相同、系数互为相反数项的多项式，问题：结果出现零系数结点。
解决方法：计算同指数项时检测系数结果是否为0，若为0则不创建结点。
在求差中同样加入该检测。

4. 求差：没有出现问题。

5. 求值：没有出现问题。

6. 销毁：注意将数组p对应位置重置为NULL，没有出现问题。

7. 清空：没有出现问题。

8. 插入新项：已添加合法性检测，没有出现问题。

9. 删除项：利用Locate函数寻找删除项位置，该项不存在则打印未找到该项，没有出现问题。

10. 修改已有项：已添加合法性检测，没有出现问题。

11. 微分:

1. 常规输入, 没有出现问题。
2. 输入负数阶数, 问题: 无法计算负阶数微分。

解决方法: 加入合法性检测。

在乘方中同样加入该检测。

12. 不定积分: 没有出现问题。

13. 定积分: 没有出现问题。

14. 乘法: 没有出现问题。

15. 乘方: 没有出现问题。

16. 除法:

1. 常规输入, 问题: 商多项式项数错误。
解决方法: 在计算过程中添加商多项式 $L \rightarrow \text{exp}$ 的自增。

2. 被除数输入0, 问题: 结果错误。

解决方法: 加入合法性检测。

3. 除数项数大于被除数项数, 问题: 结果错误。

解决方法: 加入合法性检测。

算法的时空分析

各函数时空复杂度如下: (L长度为n, La长度为a, Lb长度为b)

```
int Locate(LinkList L,int nexp);  
    //查找多项式内是否有指定的指数, 有则返回该项的位置, 没有返回0  
    //时间复杂度:  $O(n)$   
    //空间复杂度:  $O(1)$   
Status InitList(LinkList &L);  
    //在指定位置创建系数从高到低的多项式, 包括输入功能, 输入-1结束输入  
    //时间复杂度:  $O(n^2)$   
    //空间复杂度:  $O(n)$   
Status InitList(LinkList L);  
    //输出指定多项式  
    //时间复杂度:  $O(n)$   
    //空间复杂度:  $O(1)$   
LinkList AddList(LinkList La,LinkList Lb);  
    //创建一个新多项式, 内容为两个输入的多项式相加, 返回结果的头指针  
    //时间复杂度:  $O(a+b)$   
    //空间复杂度:  $O(a+b)$   
LinkList SubList(LinkList La,LinkList Lb);  
    //创建一个新多项式, 内容为两个输入的多项式相减, 返回结果的头指针  
    //时间复杂度:  $O(a+b)$   
    //空间复杂度:  $O(a+b)$   
double EvaluateList(LinkList L,double x);  
    //代入x的值计算多项式的值并返回  
    //时间复杂度:  $O(n)$   
    //空间复杂度:  $O(1)$   
Status DestroyList(LinkList &L);  
    //释放链表L包括头结点内的所有结点  
    //时间复杂度:  $O(n)$   
    //空间复杂度:  $O(1)$   
Status ClearList(LinkList &L);
```

```

//释放链表L头结点外的所有结点
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 
Status InsertNode(LinkList &L,int nexp,double ncoe);
//在链表L中按顺序插入新结点
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 
Status DeleteNode(LinkList &L,int location);
//删除链表L指定位置的结点
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 
Status EditNode(LinkList &L,int location,int nexp,double ncoe);
//修改链表L指定位置的结点
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 
LinkList DiffList(LinkList L);
//计算L的一阶导数存入新链表, 返回新链表的头指针
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(n)$ 
LinkList InteList(LinkList L);
//计算L的不定积分(常数项为0)存入新链表, 返回新链表的头指针
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(n)$ 
LinkList MultList(LinkList La,LinkList Lb);
//创建一个新多项式, 内容为两个输入的多项式相乘, 返回结果的头指针
//时间复杂度:  $O(a*b)$ 
//空间复杂度:  $O(a*b)$ 
DivResult DivList(LinkList La,LinkList Lb);
//输入被除式与除式, 以结构体形式输出商和余数的头指针
//时间复杂度:  $O(a*b)$ 
//空间复杂度:  $O(3a)$ 

```

测试结果及分析

-----多项式计算器-----

- 1、输入并创建多项式
- 2、输出多项式
- 3、求和
- 4、求差
- 5、求值
- 6、销毁
- 7、清空
- 8、修改
- 9、微分
- 10、不定积分
- 11、定积分
- 12、乘法和乘方
- 13、除法
- 0、退出程序

(之后省略菜单)

请输入操作: 1

请输入指数(输入-1结束输入): 1

请输入系数: -2

请输入指数(输入-1结束输入): 0

请输入系数: 5

请输入指数（输入-1结束输入）：3
请输入系数：1.4
请输入指数（输入-1结束输入）：-1
创建成功，多项式编号为1，按回车键返回菜单

请输入操作：1
请输入指数（输入-1结束输入）：-1
//创建多项式为0
创建成功，多项式编号为2，按回车键返回菜单

请输入操作：1
请输入指数（输入-1结束输入）：1
请输入系数：2
请输入指数（输入-1结束输入）：4
请输入系数：-2.3
请输入指数（输入-1结束输入）：-1
创建成功，多项式编号为3，按回车键返回菜单

请输入操作：2
请输入多项式编号：1
项数为：3
多项式为：
1.400 x^3 -2.000 x +5.000
输出完毕，按回车键返回菜单

请输入操作：3
请输入需要相加的第一个多项式编号：1
请输入需要相加的第二个多项式编号：3
项数为：3
多项式为：
-2.300 x^4 +1.400 x^3 +5.000
要存储该结果吗？（Y/N）：Y
存储成功，多项式编号为4，按回车键返回菜单

请输入操作：4
请输入被减数多项式编号：4
请输入减数多项式编号：3
项数为：3
多项式为：
1.400 x^3 -2.000 x +5.000
要存储该结果吗？（Y/N）：N
计算完毕，按回车键返回菜单

请输入操作：5
请输入多项式编号：1
请输入x的值：1
代入计算结果为 4.400，按回车键返回菜单

请输入操作：7
请输入多项式编号：4
清空完毕，按回车键返回菜单
请输入操作：2
请输入多项式编号：4
多项式为：0.000
//仅含头结点多项式为0
输出完毕，按回车键返回菜单

请输入操作：6

请输入多项式编号：4
销毁完毕，按回车键返回菜单
请输入操作：2
请输入多项式编号：4
该编号下无多项式！
//未找到多项式，说明正常销毁

请输入操作：8
请输入多项式编号：1
请选择操作：
1、插入新的结点
2、删除已有结点
3、修改已有结点的系数和指数
0、退出
(之后省略菜单)
1
请输入新结点的指数（输入-1取消）：5
请输入新结点的系数：2
插入成功！当前项数为：4
多项式为：
2.000 x^5 +1.400 x^3 -2.000 x +5.000
按回车键继续

请输入操作：8
请输入多项式编号：1
请选择操作：
2
请输入删除结点的指数（输入-1取消）：5
删除成功！当前项数为：3
多项式为：
1.400 x^3 -2.000 x +5.000
按回车键继续

请输入操作：8
请输入多项式编号：1
请选择操作：
3
请输入需修改结点的指数（输入-1取消）：4
未找到该项，请重新输入！
请输入需修改结点的指数（输入-1取消）：3
请输入新指数：2
请输入新系数：0.8
修改成功！当前项数为：3
多项式为：
0.800 x^2 -2.000 x +5.000
按回车键继续

请输入操作：9
请输入需要微分的多项式编号：1
请输入需求的微分阶数：2
项数为：1
多项式为：
1.600 //结果正确
要存储该结果吗？（Y/N）：N
计算完毕，按回车键返回菜单

请输入操作：10
请输入需要求不定积分的多项式编号：1

项数为: 3

多项式为:

0.267 x^3 -1.000 x^2 +5.000 x +C (C为任意常数) //结果正确

要存储该结果吗? (Y/N): N

计算完毕, 按回车键返回菜单

请输入操作: 11

请输入需要求定积分的多项式编号: 1

请输入定积分起点: 2

请输入定积分终点: 4

定积分结果为: 12.933 //结果正确

计算完毕, 按回车键返回菜单

请输入操作: 12

1. 多项式相乘

2. 多项式乘方

请输入操作: 1

请输入需要相乘的第一个多项式编号: 1

请输入需要相乘的第二个多项式编号: 3

项数为: 6

多项式为:

-1.840 x^6 +4.600 x^5 -11.500 x^4 +1.600 x^3 -4.000 x^2 +10.000 x //结果正确

要存储该结果吗? (Y/N): Y

存储成功, 多项式编号为4, 按回车键返回菜单

请输入操作: 12

1. 多项式相乘

2. 多项式乘方

请输入操作: 2

请输入需要求乘方的多项式编号: 1

请输入乘方次数: 2

项数为: 5

多项式为:

0.640 x^4 -3.200 x^3 +12.000 x^2 -20.000 x +25.000 //结果正确

要存储该结果吗? (Y/N): N

计算完毕, 按回车键返回菜单

请输入操作: 13

请输入被除数多项式编号: 4

请输入除数多项式编号: 2

除数为0

请输入操作: 13

请输入被除数多项式编号: 4

请输入除数多项式编号: 3

商项数为: 3

多项式为:

0.800 x^2 -2.000 x +5.000 //结果正确

余数多项式为: 0.000 //结果正确

要存储商吗? (Y/N): N

要存储余数吗? (Y/N): N

计算完毕, 按回车键返回菜单

请输入操作: 0

感谢您的使用, 再会!

Process exited after 342.8 seconds with return value 1

实验体会与收获

通过本实验，我复习巩固了指针和链表操作的基本功和数组操作的基本功，进一步学习了模块化程序设计的风格，开始注重人机交互界面设计和源程序的整洁性、可读性，更重视对程序健壮性的处理与输入的合法性检测，训练了程序的调试技术，开始注重程序的时空效率，为未来设计高效率算法和编写复杂程序打下基础。

栈与队列的应用 实验报告

设计目标

在C语言环境下，利用栈与队列的相关操作，完成如下程序设计：

1. 简单的行文本编辑器

建立一个空的文本编辑器，输入指令条数N，从输入文件中读入N条操作指令并执行。操作指令包括：

Move k: 将光标移动到第 k 个字符之后，如果 k=0 则将光标移到文本第一个字符之前。

Insert n \n S: 在光标后插入长度为 n 的字符串 S，光标位置不变，n≥1。

Delete n: 删除光标后的 n 个字符，光标位置不变，n≥1。

Rotate n: 反转光标后的 n 个字符，光标位置不变，n≥1。

Get: 输出此时程序内保存的文本。

Prev: 光标前移一个字符。

Next: 光标后移一个字符。

对所有执行过的Get操作，将指定的内容写入输出文件。保证输入数据合法。

2. 括号配对检验

输入表达式数目N，输入N个含有英文字母、数字、四则运算符、左右大中小括号的表达式，以'@'作为表达式的结束符，检查表达式中的左右大中小括号是否匹配，若匹配则输出"YES"，否则输出"NO"。

3. 银行业务模拟

客户业务分为两种：第一种是申请从银行得到一笔资金，即取款或借款；第二种是向银行投入一笔资金，即存款或还款。银行有两个服务窗口，相应地有两个队列。客户到达银行后先排第一个队，处理每个客户业务时，如果属于第一种，且申请额超出银行现存资金总额而得不到满足，则立刻排入第二个队等候直至满足时才离开银行；否则业务处理完后立刻离开银行，每接待完一个第二种业务的客户，则顺序检查和处理(如果可能)第二个队列中的客户，对能满足的申请者予以满足，不能满足者重新排到第二个队列的队尾。任何时刻都只开一个窗口。假设检查不需要时间，营业时间结束时所有客户立即离开银行。设计上述银行业务的事件驱动模拟系统，通过模拟方法求出客户在银行内逗留的平均时间。

第一行输入四个数 N、total、close_time、average_time，分别表示来银行的总人数、银行开始营业时拥有的款额、今天预计的营业时长和客户交易时长，之后的 N 行每行输入两个数 a、b，第一个数 a 为客户办理的款额，用负值和正值分别表示第一类和第二类业务。第二个数 b 为客户来到银行的时间。

输出前 N 行分别对应每个客户的等候时长，第 N+1 行为客户等候平均时长（保留整数）。

简单的行文本编辑器

算法设计

采用两个顺序栈PText，NText分别存储光标前的文本和光标后的文本，相应功能用栈之间的元素移动实现。

数据结构

两个顺序栈共用字符数组V[MAXSIZE]的存储空间，PText栈底为V，NText栈底为V+MAXSIZE-1。顺序栈类型定义如下：

```
typedef struct SqStack{
    char *base; //栈底
    char *top;  //栈顶
    int stacksize; //栈内元素个数
}SqStack;
```

程序结构

各函数原型、功能与接口描述如下：

```
int strcmp(char *A,char *B);
//字符串比较函数，A>B返回1，A=B返回0，A<B返回-1
int JudgeOpe(char *ope);
//判定输入的指令类型，返回指令序号
Status Move(SqStack &PText,SqStack &NText,int num);
//Move指令，将一个栈的若干栈顶元素移动到另一个栈内，使PText栈内含有num个元素
Status Prev(SqStack &PText,SqStack &NText);
//Prev指令，将PText栈顶元素出栈，放入NText
Status Next(SqStack &PText,SqStack &NText);
//Next指令，将NText栈顶元素出栈，放入PText
Status Insert(SqStack &PText,SqStack &NText,FILE *fp,int num);
//Insert指令，从文件fp读入num个字符存入PText，再将光标移动至原处
Status Get(SqStack &PText,SqStack &NText,FILE *nfp);
//Get指令，输出当前内容至文件和显示器
Status Delete(SqStack &NText,int num);
//Delete指令，将NText栈顶num个元素出栈
Status Rotate(SqStack &PText,SqStack &NText,int num);
//Rotate指令，将NText栈顶num个元素倒序放入栈内
```

调试分析

测试数据Input.txt采用输入样例数据：

```
12
Insert 13
Balanced eert
Get
Move 2
Delete 5
Get
Next
Insert 6
editor
```

```
Move 0
Get
Move 10
Rotate 4
Get
```

发现存在如下问题：

1. Insert指令无法正常执行，少读入一个末尾字符。分析发现使用fgetc()函数时忽略了上一行的换行符。

解决方法：在读入需插入的字符前先通过fgetc()函数清除换行符。

2. Move指令光标移动距离比输入值短。检查发现移动字符时循环的结束条件中包括了每次移动都会改变的PText.stacksize。

解决方法：在函数开始时定义整型ssp标记PText.stacksize的初始值，循环结束条件中使用ssp。

3. Rotate指令执行后n个数据没有反转。检查发现放入时临时数组temp[]的循环方向为从尾到头。

解决方法：修改循环方向为从头到尾。

算法的时空分析

各函数时空复杂度如下：

```
int strcmp(char *A, char *B);
//字符串比较函数，A>B返回1，A=B返回0，A<B返回-1
//A,B中较短的长度为n
//时间复杂度：O(n)
//空间复杂度：O(1)

int JudgeOpe(char *ope);
//判定输入的指令类型，返回指令序号
//指令字符串长度为n
//时间复杂度：O(n)
//空间复杂度：O(1)

Status Move(SqStack &PText, SqStack &NText, int num);
//Move指令，将一个栈的若干栈顶元素移动到另一个栈内，使PText栈内含有num个元素
//时间复杂度：O(PText.stacksize+NText.stacksize)
//空间复杂度：O(1)

Status Prev(SqStack &PText, SqStack &NText);
//Prev指令，将PText栈顶元素出栈，放入NText
//时间复杂度：O(1)
//空间复杂度：O(1)

Status Next(SqStack &PText, SqStack &NText);
//Next指令，将NText栈顶元素出栈，放入PText
//时间复杂度：O(1)
//空间复杂度：O(1)

Status Insert(SqStack &PText, SqStack &NText, FILE *fp, int num);
//Insert指令，从文件fp读入num个字符存入PText，再将光标移动至原处
//时间复杂度：O(num)
//空间复杂度：O(num)

Status Get(SqStack &PText, SqStack &NText, FILE *nfp);
//Get指令，输出当前内容至文件和显示器
//时间复杂度：O(PText.stacksize+NText.stacksize)
//空间复杂度：O(1)

Status Delete(SqStack &NText, int num);
//Delete指令，将NText栈顶num个元素出栈
//时间复杂度：O(num)
//空间复杂度：O(1)
```

```
Status Rotate(SqStack &PText, SqStack &NText, int num);  
    //Rotate指令，将NText栈顶num个元素倒序放入栈内  
    //时间复杂度: O(num)  
    //空间复杂度: O(num)
```

测试结果及分析

文件Input.txt内容:

```
12  
Insert 13  
Balanced eert  
Get  
Move 2  
Delete 5  
Get  
Next  
Insert 6  
editor  
Move 0  
Get  
Move 10  
Rotate 4  
Get
```

显示器输出:

```
Balanced eert //完成第一次插入  
Bad eert //完成光标移动与删除  
Badeditor eert //完成第二次插入  
Badeditor tree //完成光标移动与倒置  
  
-----  
Process exited after 0.08362 seconds with return value 1  
请按任意键继续. . .
```

文件Output.txt内容:

```
Badeditor tree
```

括号配对检验

算法设计

读取表达式，遇到左括号则判断其优先级是否低于栈顶左括号的优先级（'(' < '[' < '{'）；如果是，则将该括号入栈；否则认为括号配对错误。遇到右括号则判断其是否与栈顶左括号匹配，若匹配则将左括号出栈；否则认为括号配对错误。在字符串读取完毕后，判断栈是否为空，若不为空则认为括号配对错误，否则括号配对正确。

数据结构

字符数组text[MAXSIZE]存储读入的表达式，整型数组result[num]存储各表达式的判断结果，顺序栈S存储读入的左括号。顺序栈类型定义如下：

```
typedef struct SqStack{
    char *base; //栈底
    char *top; //栈顶
    int stacksize; //栈内元素个数
}SqStack;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status InitStack(SqStack S);
    //构造空顺序栈S
Status Push(SqStack S, char c);
    //c入栈
char GetTop(SqStack S);
    //取栈顶元素
char Pop(SqStack S);
    //栈顶元素出栈
Status BracketCmp(char a, char b);
    //判定括号a是否比括号b优先级低，是返回TRUE，否则返回FALSE
Status BracketPairing(char a, char b);
    //判定右括号a是否与左括号b配对，是返回TRUE，否则返回FALSE
Status CheckBracket(char *text, SqStack S);
    //检查左/右括号是否满足要求，若满足返回TRUE，否则FALSE
```

调试分析

测试数据采用输入样例数据：

```
3
2*[(x+y)/(1-x)]@
(25+x)*(a*(a+b+b))@
{1+2[c-d(7/9)]@
```

发现存在如下问题：

1. 第一个左括号无法正常入栈。
解决方法：将入栈条件修改为：栈空或栈顶括号优先级高。
2. 右括号多于左括号时无法正常判断。
解决方法：将出栈条件修改为：栈不空且左右括号匹配。

算法的时空分析

各函数时空复杂度如下：

```
Status InitStack(SqStack S);
    //构造空顺序栈S
    //时间复杂度：O(1)
    //空间复杂度：O(n)
Status Push(SqStack S, char c);
    //c入栈
    //时间复杂度：O(1)
    //空间复杂度：O(1)
char GetTop(SqStack S);
```

```

//取栈顶元素
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
char Pop(SqStack S);
//栈顶元素出栈
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status BracketCmp(char a, char b);
//判定括号a是否比括号b优先级低
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status BracketPairing(char a, char b);
//判定右括号a是否与左括号b配对
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status CheckBracket(char *text, SqStack S);
//检查左/右括号是否满足要求, 若满足返回TRUE, 否则FALSE
//时间复杂度:  $O(n)$ 
//空间复杂度:  $O(1)$ 

```

测试结果及分析

键盘输入:

```

3
2*[(x+y)/(1-x)]@
(25+x)*(a*(a+b+b))}@
{1+2[c-d(7/9)]}@

```

显示器输出:

```

YES
NO
NO

```

银行业务模拟

算法设计

读取客户信息建立客户链表customers, 建立事件链表, 并将第一个客户的到达事件插入链表。直到事件链表空为止循环处理链表的第一个事件。全部处理完毕后, 计算仍未得到处理的客户的等待时间(即一直等待到银行关门)。

处理客户到达事件时, 将当前客户插入队列Q1, 将下一个客户到达事件插入事件表; 若队列Q1只剩一个客户, 将该客户离开事件插入事件表。

处理客户离开事件时, 移除队列Q1队头客户, 判断该用户是否去队列Q2; 再判断该用户是否是存款客户, 如果是则检查Q2; 若队列Q1不空, 队列Q1中下一个客户的离开事件插入事件表。

数据结构

采取事件驱动模拟。用链表存储各客户的信息(存/取款、到达时间、离开时间), 用链表存储各事件的信息。二者结点结构定义如下:

```

typedef struct Customer{
    int Money;
    int ArriveTime;
    int LeaveTime;
    struct Customer *next;
}Customer;

typedef struct Event{
    int OccurTime; //事件的发生时间
    int EventType; //事件类型, 1表示进入, 0表示离开
    struct Event *next;
}Event,*LinkList;

```

用两个队列Q1、Q2表示两个窗口的队伍，队列结构定义如下：

```

typedef struct QNode{
    Customer *ctm;
    struct QNode *next;
}QNode;

typedef struct Queue{
    QNode *front;
    QNode *rear;
}Queue,*LinkQueue;

```

全局变量N,total,close_time,average_time存储来银行的总人数、银行开始营业时拥有的款额、今天预计的营业时长和客户交易时长。

程序结构

各函数原型、功能与接口描述如下：

```

Status InputCustomer(Customer *C,int N);
    //将客户信息输入带头结点链表
Status EnQueue(LinkQueue Q, Customer *c);
    //将客户c插入队尾
Status EnList(LinkList events,int time,int EventType);
    //按时间顺序插入新事件
int QueueLength(LinkQueue Q);
    //检查队列Q长度
Customer *De1FrontQueue(LinkQueue Q);
    //删去Q队头，返回队头客户指针
Event De1FirstEvent(LinkList events);
    //删除事件列表第一个结点，返回其事件
Status CustomerArrived(LinkList events,LinkQueue Q1, Customer *nctm);
    //客户到达事件
int CheckQ2(LinkQueue Q2,int BFTotal,int time);
    //检查Q2，处理可以取款的客户，返回Q2客户的个数
Status CustomerDeparture(LinkList events,Event pe,LinkQueue Q1,LinkQueue Q2);
    //客户离开事件

```

调试分析

测试数据采用输入样例数据：

```
4 10000 600 10
-2000 0
-11000 10
-10000 30
2000 50
```

发现存在如下问题：

1. 不能输出一一直在等待的客户的等待时间。

解决方法：在最后加入判断，若客户未离开则计算其等待时间。

2. 检查Q2时花费了多余时间检查。

解决方法：检查Q2时若银行资金总额小于初始值则停止循环。

算法的时空分析

各函数时空复杂度如下：

```
Status InputCustomer(Customer *C,int N);
    //将客户信息输入带头结点链表
    //时间复杂度: O(N)
    //空间复杂度: O(N)
Status EnQueue(LinkQueue Q, Customer *c);
    //将客户c插入队尾
    //时间复杂度: O(1)
    //空间复杂度: O(1)
Status EnList(LinkList events,int time,int EventType);
    //按时间顺序插入新事件
    //时间复杂度: O(events.length)
    //空间复杂度: O(1)
int QueueLength(LinkQueue Q);
    //检查队列Q长度
    //时间复杂度: O(Q.length)
    //空间复杂度: O(1)
Customer *DelFrontQueue(LinkQueue Q);
    //删去Q队头, 返回队头客户指针
    //时间复杂度: O(1)
    //空间复杂度: O(1)
Event DelFirstEvent(LinkList events);
    //删除事件列表第一个结点, 返回其事件
    //时间复杂度: O(1)
    //空间复杂度: O(1)
Status CustomerArrived(LinkList events, LinkQueue Q1, Customer *nctm);
    //客户到达事件
    //时间复杂度: O(events.length)
    //空间复杂度: O(1)
int checkQ2(LinkQueue Q2,int BFTotal,int time);
    //检查Q2, 处理可以取款的客户, 返回Q2客户的个数
    //时间复杂度: O(Q2.length)
    //空间复杂度: O(1)
Status CustomerDeparture(LinkList events, Event pe, LinkQueue Q1, LinkQueue Q2);
    //客户离开事件
    //时间复杂度: O(event.length)
    //空间复杂度: O(1)
```

测试结果及分析

键盘输入：

```
4 10000 600 10
-2000 0
-11000 10
-10000 30
2000 50
```

显示器输出：

```
0
590
30
0
155
```

实验体会与收获

通过本实验，我进一步了解了栈的后进先出、队列的先进先出的特点，掌握了栈、队列的表示和实现技术，掌握如何运用栈、队列的特点来构建算法，了解了事件驱动模拟的算法构建。

二叉树的应用 实验报告

设计目标

在C++环境下，利用二叉树与Huffman树的相关操作，实现对任意文件的压缩处理，压缩之后的文件后缀名为huff，同时可以对所有后缀名为huff的压缩文件进行解压缩。

以1个字节为单位进行Huffman编码，要求可以完成压缩和解压的步骤，并且解压出来的文件没有任何损失。

算法设计

压缩文件时执行的操作：

1. 读入文件，统计每种字节出现的次数（256种）
2. 根据字节出现的次数创建Huffman树
3. 由Huffman树得到各字节的Huffman编码
4. 将压缩前文件类型、各字节出现的次数信息写入.huff文件
5. 读入文件，按字节将对应的编码写入.huff文件，缺位用0补齐

解压缩文件时执行的操作：

1. 读入.huff文件，根据文件开头的信息获得Huffman树
2. 根据Huffman编码，将对应的字节写入文件

.huff文件结构为：

```
char FileType[FILE_TYPE_LENGTH] //源文件类型（,xxx）
int FileSize //源文件大小（用于解压时确定是否解码结束）
int count[256] //各字节按顺序的出现次数
//Huffman码比特流
```

基于贪心策略构建Huffman树的算法思想概括如下：

1. 给定n个权值作为n个叶子节点，构成初始的森林 $F=\{T_1, T_2, \dots, T_n\}$ ，其中每棵二叉树 T_i 仅有一个权值为 W_i 的根节点。
2. 在森林F中选取两棵根节点权值最小的树作为左右子树，构造一棵新的二叉树，并置新二叉树根节点的权值为其左右子树上根节点的权值之和。
3. 从森林F中删除这两棵被选中的树，同时将新构造的二叉树加入到森林F中。
4. 重复步骤2和步骤3，直到森林F中只含一棵二叉树为止，即得到Huffman树。

数据结构

采用结构体数组FileByteInfo[256]存储各字节（无符号字符）的信息，下标即表示无符号字符的值，结构体定义为：

```
typedef struct Info{
    int count;    //无符号字符出现的次数
    char *HCode;  //字符对应的Huffman编码
}Info;           //文件内无符号字符的信息
```

Huffman编码过程借助Huffman树实现，Huffman树采用顺序存储，其结点结构定义为：

```
typedef struct HTNode{
    unsigned char data;
    int weight;      //权重
    int lchild, rchild;
    int parent;
}HTNode, *HuffmanTree;
```

程序结构

各函数原型、功能与接口描述如下：

```
int menu();
    //输出菜单，返回操作
Status CountByte(FILE *OriginFile, Info *FileByteInfo, int &types, int &FileSize);
    //统计字符出现次数
Status FindSmallest(HuffmanTree T, int i, int &s1, int &s2);
    //找到数组中权重最小且未被连接过的两个结点
Status InitHuffmanTree(HuffmanTree &T, Info *FileByteInfo, int types);
    //建立Huffman树（顺序存储类型）
Status HuffmanCoding(HuffmanTree T, Info *FileByteInfo, int types);
    //根据Huffman树得到各字符的Huffman编码
Status ReNaming(char *FileName, char *FileType, int mode);
    //更改文件后缀
    //mode 1为更改为.huff后缀
    //mode 2为将.huff后缀改回原后缀
Status WriteInfo(FILE *NewFile, Info *FileByteInfo);
    //将各字节的出现次数写入文件
Status WriteHuffmanCode(FILE *OriginFile, FILE *NewFile, Info *FileByteInfo);
    //按源文件写入对应的Huffman编码
Status GetInfo(FILE *OriginFile, Info *FileByteInfo, int &types);
    //从文件中读入字符数信息
Status HuffmanDecoding(HuffmanTree T, int types, int FileSize, FILE
*OriginFile, FILE *NewFile);
    //按Huffman编码还原源文件
```

```
Status HuffmanDecompress(Info *FileByteInfo);  
    //解压过程
```

调试分析

测试数据Input.txt采用样例文件：2_1.txt（英文文本），2_2.txt（中文文本），2_3.bmp（图片），2_4.mp4（视频），2_5.exe（可执行文件）。

发现存在如下问题：

1. 解码时无法正常结束，无法确定末尾的0是否是补位产生的。
解决方法：存储FileSize记录源文件大小，确定解码结束位置。
2. 解码时需要确定源文件类型。
解决方法：将原文件类型以字符串形式存入.huff文件开头。

算法的时空分析

各函数时空复杂度如下：

```
int menu();  
    //输出菜单，返回操作  
    //时间复杂度：O(1)  
    //空间复杂度：O(1)  
Status CountByte(FILE *OriginFile, Info *FileByteInfo, int &types, int &FileSize);  
    //统计字符出现次数  
    //时间复杂度：O(n)  
    //空间复杂度：O(1)  
Status FindSmallest(HuffmanTree T, int i, int &s1, int &s2);  
    //找到数组中权重最小且未被连接过的两个结点  
    //时间复杂度：O(n)  
    //空间复杂度：O(1)  
Status InitHuffmanTree(HuffmanTree &T, Info *FileByteInfo, int types);  
    //建立Huffman树（顺序存储类型）  
    //时间复杂度：O(n)  
    //空间复杂度：O(n)  
Status HuffmanCoding(HuffmanTree T, Info *FileByteInfo, int types);  
    //根据Huffman树得到各字符的Huffman编码  
    //时间复杂度：O(logn)  
    //空间复杂度：O(1)  
Status ReNaming(char *FileName, char *FileType, int mode);  
    //更改文件后缀  
    //mode 1为更改为.huff后缀  
    //mode 2为将.huff后缀改回原后缀  
    //时间复杂度：O(1)  
    //空间复杂度：O(1)  
Status WriteInfo(FILE *NewFile, Info *FileByteInfo);  
    //将各字节的出现次数写入文件  
    //时间复杂度：O(1)  
    //空间复杂度：O(1)  
Status WriteHuffmanCode(FILE *OriginFile, FILE *NewFile, Info *FileByteInfo);  
    //按源文件写入对应的Huffman编码  
    //时间复杂度：O(n)  
    //空间复杂度：O(1)  
Status GetInfo(FILE *OriginFile, Info *FileByteInfo, int &types);  
    //从文件中读入字符数信息
```

```
//时间复杂度: O(1)
//空间复杂度: O(1)
Status HuffmanDecoding(HuffmanTree T,int types,int FileSize,FILE
*OriginFile,FILE *NewFile);
//按Huffman编码还原源文件
//时间复杂度: O(n)
//空间复杂度: O(1)
Status HuffmanDecompress(Info *FileByteInfo);
//解压过程
```

测试结果及分析

对五个文件，均可以在相同根目录下生成同名.huff文件，并可以独立从.huff文件解压生成源文件，其在文件资源管理器中的状态如下图：

2_5.huff	2024/12/22 21:43	HUFF 文件	34 KB
2_5.exe	2023/11/2 16:56	应用程序	53 KB
2_4.mp4	2023/11/2 16:43	MP4 文件	128 KB
2_4.huff	2024/12/22 21:43	HUFF 文件	128 KB
2_3.huff	2024/12/22 21:43	HUFF 文件	35 KB
2_3.bmp	2024/12/22 21:44	BMP 文件	142 KB
2_2.txt	2023/11/2 16:19	文本文档	1 KB
2_2.huff	2024/12/22 21:43	HUFF 文件	2 KB
2_1.txt	2023/11/2 16:19	文本文档	1 KB
2_1.huff	2024/12/22 21:43	HUFF 文件	2 KB

平均压缩率为61.8%，最优压缩率为24.6%。

实验体会与收获

通过本实验，我掌握了二叉树的递归特性，了解了二叉树的常用存储结构、创建及遍历等基本运算，并掌握了如何应用树的结构解决具体问题。

图 实验报告

设计目标

在C++环境下，利用图的相关操作，完成如下程序设计：

1. 图的遍历

输入一个无向图，输出图的深度优先搜索遍历顺序与广度优先搜索遍历顺序（要求：当有多个节点可以搜索时，优先去节点编号最小的那个）。

输入格式：第一行两个数 n, m ($1 < n < 30, 1 < m < 300$)，分别表示顶点数量和边的数量。接下来的 m 行每行输入两个数 a, b ，表示顶点 a 与顶点 b 之间有边相连，顶点编号从 1 到 n 。最后输入一个数 s 表示遍历的起始顶点编号。

输出格式：输出两行序列，第一行为深度优先搜索遍历的顺序，第二行为广度优先搜索遍历的顺序。

2. 求通讯网的最小代价生成树

输入一个无向通讯网图，用 Prim 和 Kruskal 算法计算最小生成树并输出。

输入格式：第一行是两个数 n, m ($1 < n < 10000, 1 < m < 100000$)，分别表示顶点数量和边的数量。接下来的 m 行每行输入三个数 a, b, w ；表示顶点 a 与顶点 b 之间有代价为 w 的边相连，顶点编号从 1 到 n 。

输出格式：一个数，即最小生成树的各边的长度之和。

3. 铁路交通网的最短路径

输入一个无向铁路交通图、始发站和终点站，用 Dijkstra 算法计算从始发站到终点站的最短路径。

输入格式：第一行是两个数 n, m ($1 < n < 100000, 1 < m < 1000000$)，分别表示顶点数量和边的数量。接下来的 m 行每行输入三个数 a, b, w ；表示顶点 a 与顶点 b 之间有长度为 w 的边相连，顶点编号从 1 到 n 最后输入两个数 s, t 表示遍历的起始顶点编号和终点编号。

输出格式：一个数，为从起点到终点的最短路径长度。

4. 显示图

可以直接使用题目 1 的输入数据画出图的情况则为满足要求，要求图整体美观，可以体现出顶点的编号，边的交叉尽量少。

图的遍历

算法设计

深度优先遍历（DFS）采用递归算法，算法思想如下：

1. 初始化访问情况数组，从起始节点开始，将其标记为已访问，以避免重复访问。
2. 对于起始节点的每一个未访问的邻接节点，递归地调用深度优先遍历函数。这意味着每次找到一个新的邻接节点，就将其视为新的起始节点，继续深入探索它的邻接节点。
3. 当一个节点的所有邻接节点都被访问后，递归函数会返回上一层调用。回溯到上一层后，继续探索上一层节点的其他未访问邻接节点。
4. 当所有节点都被访问过，或者没有更多的邻接节点可以探索时，递归过程结束，从小到大对未访问结点再次进行如上操作。

广度优先遍历（BFS）借助队列实现，算法思想如下：

1. 初始化访问情况数组，从起始节点开始，将其标记为已访问，以避免重复访问。
2. 访问起始结点并将起始节点入队。
3. 直到队列为空为止循环：取队列头节点，从小到大访问其未访问的邻接节点，并将它们入队。
4. 循环结束，从小到大对未访问结点再次进行如上操作。

数据结构

采用邻接矩阵存储无向图的各边信息。图与邻接矩阵类型定义如下：

```
typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;
```

在BFS中需要用到队列，链队列结点与队列类型定义如下：

```
typedef struct QNode{
    int data;
    struct QNode *next;
}QNode;

typedef struct Queue{
    QNode *front;
    QNode *rear;
}LinkQueue;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
Status EnQueue(LinkQueue &Q,int data)
    //入队操作
Status DeQueue(LinkQueue &Q,int &data)
    //出队操作
Status QueueEmpty(LinkQueue Q)
    //判断队列是否为空
Status DFSTraverse(MGraph G,int start)
    //深度优先搜索
void DFS(MGraph G,int u,int *visited)
    //深度优先搜索的递归算法
Status BFSTraverse(MGraph G,int start)
    //广度优先搜索
void BFS(MGraph G,int u,int *visited,LinkQueue &Q)
    //广度优先搜索利用队列的循环算法
```

注意本题需要从输入的起点出发遍历，故在搜索中先调用起点的DFS和BFS，再从顶点1开始对未访问的顶点依次调用。

调试分析

测试数据Input.txt采用输入样例数据：

```
5 6
1 5
1 4
3 1
2 1
2 5
5 3
1
```

未发现明显问题。

算法的时空分析

各函数时空复杂度如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
```

```

//时间复杂度:  $O(\text{arcnum})$ 
//空间复杂度:  $O(1)$ 
Status EnQueue(LinkQueue &Q, int data)
//入队操作
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status DeQueue(LinkQueue &Q, int &data)
//出队操作
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status QueueEmpty(LinkQueue Q)
//判断队列是否为空
//时间复杂度:  $O(1)$ 
//空间复杂度:  $O(1)$ 
Status DFSTraverse(MGraph G, int start)
//深度优先搜索
void DFS(MGraph G, int u, int *visited)
//深度优先搜索的递归算法
//时间复杂度:  $O(\text{vexnum}^2)$ 
//空间复杂度:  $O(1)$ 
Status BFSTraverse(MGraph G, int start)
//广度优先搜索
void BFS(MGraph G, int u, int *visited, LinkQueue &Q)
//广度优先搜索利用队列的循环算法
//时间复杂度:  $O(\text{vexnum}^2)$ 
//空间复杂度:  $O(1)$ 

```

测试结果及分析

输入内容:

```

5 6
1 5
1 4
3 1
2 1
2 5
5 3
1

```

显示器输出:

```

1 2 5 3 4
1 2 3 4 5

```

求通讯网的最小代价生成树

算法设计

Prim算法的基本思想是从一个起始顶点开始, 逐步扩展生成树, 直到包含所有顶点, 算法思想如下:

1. 从连通无向网G中选择一个起始顶点 u_0 , 首先将它加入到集合T中。
2. 在集合T中的顶点u与不在T中的顶点v之间选择权值最小的边(u, v), 将顶点v加入到集合T中。
3. 重复步骤2, 直到网络中的所有顶点都加入到生成树顶点集合T中为止。

Kruskal算法的基本思想是从边的角度出发，逐步构建最小生成树。算法思想如下：

1. 连通网 $G = (V, E)$ ，令最小生成树的初始状态为只有 n 个顶点而无边的非连通图 $T = (V, \{\})$ ，即图中每个顶点自成一个连通分量。
2. 在 E 中选择权值最小的边，若该边依附的顶点落在 T 中不同的连通分量上（即不形成回路），则将此边加入到 T 中，否则舍去此边而选择下一条权值最小的边。
3. 重复上述步骤，直到 T 中所有顶点都在同一连通分量上为止。

数据结构

采用邻接矩阵存储无向图的各边信息。图与邻接矩阵类型定义如下：

```
typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;
```

在Prim算法中用closedge数组标记各顶点到当前生成树的最短距离，其结构体定义如下：

```
typedef struct closedge{
    int adjvex;
    int lowcost;
}closedge;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
int MiniSpanTree_PRIM(MGraph G)
    //从顶点1出发用Prim算法构建最小生成树
    //返回树各边的长度之和
int MiniSpanTree_KRUS(MGraph G)
    //从顶点1出发用Kruskal算法构建最小生成树
    //返回树各边的长度之和
```

调试分析

测试数据采用输入样例数据：

```
4 5
1 2 2
1 3 2
1 4 3
2 3 4
3 4 3
```

未发现明显问题。

算法的时空分析

各函数时空复杂度如下：


```

Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
    //时间复杂度: O(arcnum)
    //空间复杂度: O(1)

int MiniSpanTree_PRIM(MGraph G)
    //从顶点1出发用Prim算法构建最小生成树
    //返回树各边的长度之和
    //时间复杂度: O(vexnum^2)
    //空间复杂度: O(vexnum)

int MiniSpanTree_KRUS(MGraph G)
    //从顶点1出发用Kruskal算法构建最小生成树
    //返回树各边的长度之和
    //时间复杂度: O(arcnum*log(arcnum))
    //空间复杂度: O(vexnum)

```

测试结果及分析

键盘输入：

```

4 5
1 2 2
1 3 2
1 4 3
2 3 4
3 4 3

```

显示器输出：

```

7

```

铁路交通网的最短路径

算法设计

Dijkstra算法通过贪心策略，不断选择当前路径代价最小的节点进行更新，逐步扩展搜索范围，直到找到从源节点到所有可达节点的最短路径。算法描述如下：

1. 从起点开始，将起点的距离设置为0，其他节点的距离设置为无穷大，表示还未找到到达该节点的路径。
2. 从未访问的节点中，选择距离起点最近的节点，然后将其标记为已访问。
3. 对选中节点的所有邻居节点，检查从起点经过当前节点到达邻居节点的路径是否比当前已知最短路径更短，如果是，则更新邻居节点的最短路径值。
4. 重复步骤2和步骤3，直到所有节点都被访问，或剩余未访问节点与起点不连通。
5. 算法完成时，每个节点的最短路径长度都已确定。

数据结构

采用邻接矩阵存储无向图的各边信息。图与邻接矩阵类型定义如下：

```

typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;

```

用closedge数组标记各顶点到当前连通分量的最短距离，其结构体定义如下：

```
typedef struct closedge{
    int adjvex;
    int lowcost;
}closedge;
```

程序结构

各函数原型、功能与接口描述如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
int ShortestPath_DIJ(MGraph G,int start,int end)
    //用Dijkstra算法计算start到end的最短路径
    //返回最短路径的长度
```

调试分析

测试数据采用输入样例数据：

```
4 5
1 2 10
1 3 20
2 3 15
2 4 30
3 4 20
1 4
```

未发现明显问题。

算法的时空分析

各函数时空复杂度如下：

```
Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
    //时间复杂度：O(arcnum)
    //空间复杂度：O(1)
int ShortestPath_DIJ(MGraph G,int start,int end)
    //用Dijkstra算法计算start到end的最短路径
    //返回最短路径的长度
    //时间复杂度：O(vexnum^2)
    //空间复杂度：O(vexnum)
```

测试结果及分析

键盘输入：

```
4 5
1 2 10
1 3 20
2 3 15
2 4 30
3 4 20
1 4
```

显示器输出：

```
40
```

显示图

算法设计

希望边的交叉尽量少，采用力导向算法。假设有边的顶点之间存在弹簧弹力，满足 $F = K_1(r - r_0)$ ；每个顶点直接存在库仑斥力，满足 $F = \frac{K_2}{r^2}$ 。将 K_1, K_2, r_0 均视为手动调整的超参数，初始时将顶点均匀安排在圆周上，用受力模拟调整各顶点位置。

数据结构

采用邻接矩阵存储无向图的各边信息。图与邻接矩阵类型定义如下：

```
typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;
```

矢量结构体定义如下：

```
typedef struct Vector{
    double x;
    double y;
}Vector;
```

各顶点的坐标与速度、加速度信息定义如下：

```
typedef struct Vertex{
    double x;
    double y;
    Vector v;
    Vector a;
}Vertex;
```

程序结构

各函数原型、功能与接口描述如下：

```

Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
void GetA(MGraph G,Vertex *ver)
    //计算各顶点的加速度
void GetNextStatus(MGraph G,Vertex *ver)
    //计算下一时刻的速度与坐标
void PrintGraph(MGraph G,Vertex *ver)
    //输出图

```

调试分析

测试数据采用输入样例数据：

```

5 6
1 5
1 4
3 1
2 1
2 5
5 3
1

```

需要多次手动调整各超参数。最终确定：

```

#define T_0 0.00001 //间隔时间
#define X_0 100 //弹簧0形变距离
#define K_1 0.5
#define K_2 0.00001

```

算法的时空分析

各函数时空复杂度如下：

```

Status SetArcs(MGraph &G)
    //输入各边，存储到邻接矩阵
    //时间复杂度：O(arcnum)
    //空间复杂度：O(1)
void GetA(MGraph G,Vertex *ver)
    //计算各顶点的加速度
    //时间复杂度：O(vexnum^2)
    //空间复杂度：O(1)
void GetNextStatus(MGraph G,Vertex *ver)
    //计算下一时刻的速度与坐标
    //时间复杂度：O(vexnum)
    //空间复杂度：O(1)
void PrintGraph(MGraph G,Vertex *ver)
    //输出图
    //时间复杂度：O(vexnum+arcnum)
    //空间复杂度：O(1)

```

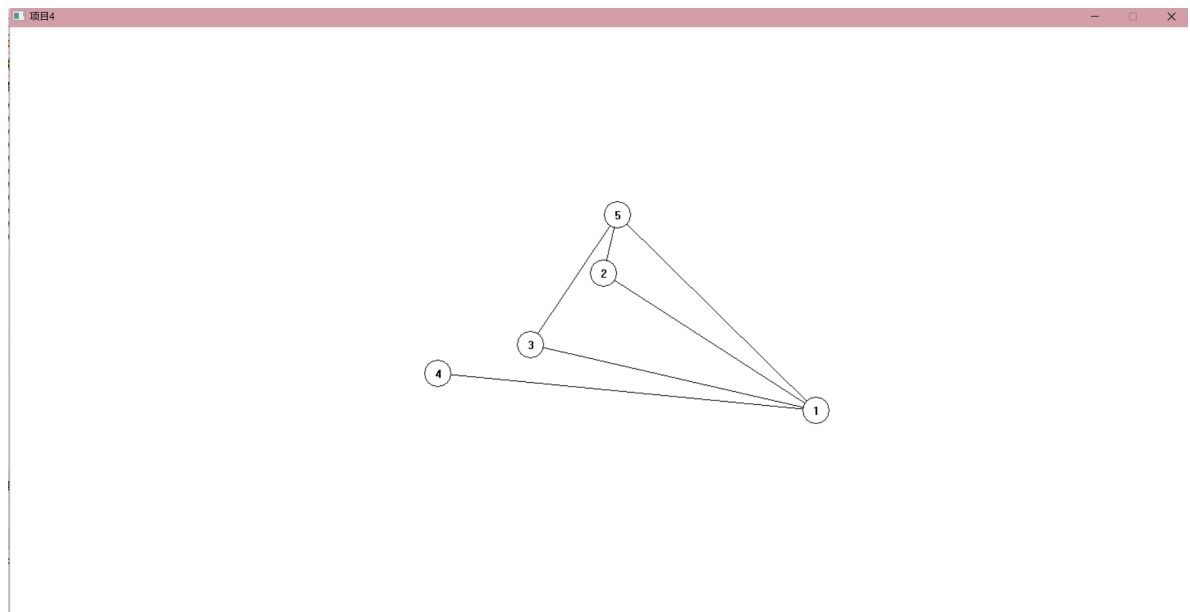
测试结果及分析

键盘输入：

```
5 6
1 5
1 4
3 1
2 1
2 5
5 3
```

显示器输出：

```
1 262.621448 106.274993
2 3.501857 -61.804436
3 -86.609065 26.129811
4 -199.993378 61.408305
5 20.479120 -132.008660 //各顶点坐标
```



实验体会与收获

通过本实验，我掌握了图的两种存储结构：邻接矩阵表示法和邻接表表示法；掌握了图的DFS遍历和BFS遍历的算法；学会了利用图的模型来编程解决实际应用问题。

哈希表 实验报告

姓名：时分

学号：PB23151799

设计目标

在C++环境下，完成以下任务：

1. 输入关键字序列；
2. 用除留余数法构建哈希函数，用线性探测法解决冲突，构建哈希表 HT1；
3. 用除留余数法构建哈希函数，用拉链法解决冲突，构建哈希表 HT2；
4. 分别对 HT1 和 HT2 计算在等概率情况下查找成功和查找失败的 ASL；

5. 分别在 HT1 和 HT2 中查找给定的关键字，给出比较次数。

测试文件中包括：关键字个数 n ，关键字 key （这里我们认为关键字 key 就是哈希表中元素对应的哈希函数值），和除留余数法中的 p 。要求按顺序输入关键字后，输出哈希表的相关信息，包括表的结构，成功查找次数，失败查找次数，平均查找长度等信息。

算法设计

用除留余数法构建的哈希函数为：

$$H(key) = key \mod p, p \leq m$$

此题中直接取哈希表长 $m=p$ 。

线性探测法求得第 i 个地址为：

$$H_i = (H(key) + i) \mod m, i = 1, 2, \dots, k (k \leq m - 1)$$

拉链法直接采用链表，在同一地址上存储多个记录。

查找时直接利用关键字 key 求出地址 $H(key)$ ，在 HT1 上利用线性探测寻找该记录，成功查找次数 S 即为线性探测的次数 i ，若遇到空位置说明查找失败，失败查找次数 U 即为空位置与初始位置的距离；在 HT2 上遍历该地址上的链表，成功查找次数 S 即为该记录在链表上的位置，遍历完仍未找到说明查找失败，失败查找次数 U 即为链表长度+1。

假设各记录被查找概率相等即可计算出 ASL：

$$ASL = \frac{1}{n} \sum_{i=1}^n S_i$$

数据结构

线性探测法构建的 HT1 结构定义如下：

```
typedef struct HTable{
    int *elem;    //存储元素
    int *successtime; //存储每个元素的成功查找次数
    int *failtime;  //存储每个元素的失败查找次数
    int count;     //记录个数
}HashTable;
```

拉链法构建的 HT2 结构定义如下：

```
typedef struct CNode{
    int elem;
    struct CNode *next;
}CNode,*Chain;

typedef struct CHTable{
    Chain *ChainHash;
    int *ChainLength; //存储每个链表的长度
    int count;        //记录个数
}CHashTable;
```

采用全局变量 $tablesize=p$ 标记线性探测法中的 p 和表长 m ，全局变量 num 标记记录总数。

程序结构

各函数原型、功能与接口描述如下：

```
int Hash(int x)
    //计算哈希地址
    //时间复杂度: O(1)
    //空间复杂度: O(1)
Status InitTable(HashTable &HT, CHashTable &CHT, int *data)
    //创建并填写哈希表
    //线性探测法时间复杂度: O(n^2)
    //拉链法时间复杂度: O(n)
    //空间复杂度: O(n)
Status PrintTable(HashTable HT, CHashTable CHT)
    //输出哈希表相关信息
    //时间复杂度: O(1)
    //空间复杂度: O(1)
```

调试分析

测试数据Input.txt采用样例文件：hush_1.txt至hush_6.txt。

发现存在如下问题：

1. 数据量太大，横排输出会导致内容叠加，难以辨别。

解决方案：调整为竖排输出。

算法的时空分析

各函数时空复杂度如下：

```
int Hash(int x)
    //计算哈希地址
Status InitTable(HashTable &HT, CHashTable &CHT, int *data)
    //创建并填写哈希表
Status PrintTable(HashTable HT, CHashTable CHT)
    //输出哈希表相关信息
```

测试结果及分析

对hush1.txt处理的输入、输出如下：

请输入文件路径：hush1.txt

HT1内容为：

表中的关键字：

0	-
1	1
2	79
3	107
4	4
5	118
6	56
7	72
8	-
9	48

10	87
11	126
12	-

成功查找次数:

0	-
1	1
2	2
3	1
4	1
5	5
6	3
7	1
8	-
9	1
10	2
11	3
12	-

失败查找次数:

0	1
1	8
2	7
3	6
4	5
5	4
6	3
7	2
8	1
9	4
10	3
11	2
12	1

查找成功的平均查找长度: 2.00

查找失败的平均查找长度: 3.62

HT2内容为:

表中的关键字:

0	-		
1	1	79	118
2	-		
3	107		
4	4	56	
5	-		
6	-		
7	72		
8	-		
9	48	87	126
10	-		
11	-		
12	-		

成功查找次数:

0	0		
1	1	2	3
2	0		
3	1		
4	1	2	
5	0		
6	0		
7	1		


```
8    0
9    1    2    3
10   0
11   0
12   0
```

失败查找次数:

```
0    1
1    4
2    1
3    2
4    3
5    1
6    1
7    2
8    1
9    4
10   1
11   1
12   1
```

查找成功的平均查找长度: 1.70

查找失败的平均查找长度: 1.77

实验体会与收获

通过本实验，我掌握了哈希表的定义和特点；掌握了哈希函数的构造方法和解决冲突的技术，实现哈希造表；掌握了哈希表的查找技术；掌握哈希表平均查找长度ASL的计算方法。

附录：各实验源代码

稀疏一元多项式运算器

```
#include<stdio.h>
#include<windows.h>
#define TRUE 1
#define FALSE 0
#define OK 1
#define ERROR 0
#define INFEASIBLE -1
#define OVERFLOW -2
#define NUM 50
typedef int Status;

typedef struct LNode{
    double coef;
    int exp;
    struct LNode *next;
}LNode,*LinkList;

/*typedef struct SNode{
    char data;
    LinkList list;
    struct SNode *next;
}SNode,*LinkStack;*/
```

```

typedef struct DivResult{
    LinkList quotient;
    LinkList remainder;
}DivResult;//用结构体形式定义除法的结果，包括两个链表头指针，分别是商和余数

int Locate(LinkList L,int nexp){
    //查找多项式内是否有指定的指数，有则返回该项的位置，没有返回0
    LNode *p=L->next;
    int i=1;
    while(p){
        if(p->exp == nexp) return i;
        p=p->next;i++;
    }
    return 0;
}

Status InitList(LinkList L){
    //在指定位置创建系数从高到低的多项式
    LNode *p=NULL,*q=NULL;
    int nexp;
    double ncoe;
    int Locate(LinkList L,int nexp);
    while(1){
        printf("请输入指数（输入-1结束输入）：");
        scanf("%d",&nexp);
        getchar();
        while(Locate(L,nexp)){
            printf("指数重复，请重新输入:(输入-1结束输入):");
            scanf("%d",&nexp);
            getchar();
        }
        while(nexp < -1){
            printf("指数不合法，请重新输入:(输入-1结束输入):");
            scanf("%d",&nexp);
            getchar();
        }
        if(nexp == -1) break;
        printf("请输入系数：");
        scanf("%lf",&ncoe);
        getchar();
        if(ncoe != 0){
            p=(LNode*)malloc(sizeof(LNode));
            if(p==NULL){
                printf("内存错误!\n");
                exit(OVERFLOW);
            }
            p->exp = nexp;
            p->coef = ncoe;
            p->next = NULL;//完成新结点创建和输入
            q=L;
            while(q->next && q->next->exp > p->exp){
                q=q->next;
            }
            p->next = q->next;
            q->next = p;
            //向链表中按顺序插入新结点
            L->exp++;
        }
    }
}

```

```

    }
    //系数为0时不创建新结点，用仅含头结点的链表表示多项式0
}
return OK;
}

Status PrintList(LinkList L){
    //输出多项式
    LNode *p=L->next;
    if(p){
        printf("项数为: %d\n",L->exp);
        printf("多项式为: \n");
        while(p->next){
            if(p->exp != 1){
                if(p->next->coef > 0)
                    printf("%.31f x^%d +",p->coef,p->exp);
                else
                    printf("%.31f x^%d ",p->coef,p->exp);
            }
            else{
                if(p->next->coef > 0)
                    printf("%.31f x +",p->coef,p->exp);
                else
                    printf("%.31f x ",p->coef,p->exp);
            }
            p=p->next;
        }
        if(p->exp != 0 && p->exp != 1)
            printf("%.31f x^%d ",p->coef,p->exp);
        else if(p->exp == 1)
            printf("%.31f x ",p->coef);
        else printf("%.31f ",p->coef);
    }
    else{
        printf("多项式为: 0.000 ");
    }//分开考察多项式为0的情况
    return OK;
}

LinkList AddList(LinkList La,LinkList Lb){
    //创建一个新多项式，内容为两个输入的多项式相加，返回结果的指针
    LinkList Lc=(LinkList)malloc(sizeof(LNode));
    if(Lc==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    Lc->exp=0;
    Lc->next = NULL;
    LNode *p=NULL,*q=Lc;
    LNode *pa=La->next,*pb=Lb->next;
    while(pa && pb){
        p=(LNode*)malloc(sizeof(LNode));
        if(p==NULL){
            printf("内存错误!\n");
            exit(OVERFLOW);
        }
        if(pa->exp > pb->exp) {
            p->exp = pa->exp;

```

```

        p->coef = pa->coef;
        p->next = NULL;
        q->next = p;
        q = p;
        pa = pa->next;
        Lc->exp++;
    }
    else if(pa->exp < pb->exp) {
        p->exp = pb->exp;
        p->coef = pb->coef;
        p->next = NULL;
        q->next = p;
        q = p;
        pb = pb->next;
        Lc->exp++;
    } //将指数较大的项放入Lc
    else{
        if(pa->coef + pb->coef > 0.00001 || pa->coef + pb->coef < -0.00001){
            //即pa与pb系数相加不为0，考虑浮点数的运算误差
            p->exp = pa->exp;
            p->coef = pa->coef + pb->coef;
            p->next = NULL;
            q->next = p;
            q = p;
            pa = pa->next;
            pb = pb->next;
            Lc->exp++;
        }
        else{
            pa = pa->next;
            pb = pb->next;
            free(p);
        } //若出现指数相等的项，计算相加后系数是否为0，再加入Lc
    }
}

while(pa){
    p=(LNode*)malloc(sizeof(LNode));
    if(p==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    p->exp = pa->exp;
    p->coef = pa->coef;
    p->next = NULL;
    q->next = p;
    q = p;
    pa = pa->next;
    Lc->exp++;
}

while(pb){
    p=(LNode*)malloc(sizeof(LNode));
    if(p==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    p->exp = pb->exp;
    p->coef = pb->coef;
    p->next = NULL;

```

```

        q->next = p;
        q = p;
        pb = pb->next;
        Lc->exp++;
    } //将La, Lb中剩余的项加入Lc
    return Lc;
}

LinkList SubList(LinkList La, LinkList Lb){
    //创建一个新多项式，内容为两个输入的多项式相减，返回结果的指针
    //内容与加法基本相同，只将pb系数改为负数
    LinkList Lc=(LinkList)malloc(sizeof(LNode));
    if(Lc==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    Lc->exp=0;
    Lc->next = NULL;
    LNode *p=NULL, *q=Lc;
    LNode *pa=La->next, *pb=Lb->next;
    while(pa && pb){
        p=(LNode*)malloc(sizeof(LNode));
        if(p==NULL){
            printf("内存错误!\n");
            exit(OVERFLOW);
        }
        if(pa->exp > pb->exp) {
            p->exp = pa->exp;
            p->coef = pa->coef;
            p->next = NULL;
            q->next = p;
            q = p;
            pa = pa->next;
            Lc->exp++;
        }
        else if(pa->exp < pb->exp) {
            p->exp = pb->exp;
            p->coef = -pb->coef;
            p->next = NULL;
            q->next = p;
            q = p;
            pb = pb->next;
            Lc->exp++;
        }
        else{
            if(pa->coef - pb->coef > 0.00001 || pa->coef - pb->coef < -0.00001){
                //即pa与pb系数相加不为0，考虑浮点数的运算误差
                p->exp = pa->exp;
                p->coef = pa->coef - pb->coef;
                p->next = NULL;
                q->next = p;
                q = p;
                pa = pa->next;
                pb = pb->next;
                Lc->exp++;
            }
            else{
                pa = pa->next;
            }
        }
    }
}

```

```

        pb = pb->next;
        free(p);
    }
}

while(pa){
    p=(LNode*)malloc(sizeof(LNode));
    if(p==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    p->exp = pa->exp;
    p->coef = pa->coef;
    p->next = NULL;
    q->next = p;
    q = p;
    pa = pa->next;
    Lc->exp++;
}

while(pb){
    p=(LNode*)malloc(sizeof(LNode));
    if(p==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    p->exp = pb->exp;
    p->coef = -pb->coef;
    p->next = NULL;
    q->next = p;
    q = p;
    pb = pb->next;
    Lc->exp++;
}

return Lc;
}

double EvaluateList(LinkList L,double x){
    //代入x的值计算多项式的值并返回
    double sum = 0,pow;
    int i;
    LNode *p = L->next;
    while(p){
        pow = 1;
        for(i=0;i < p->exp;i++)
            pow *= x;
        sum += p->coef * pow;
        p = p->next;
    }
    return sum;
}

Status DestroyList(LinkList L){
    //释放L包括头结点内的所有结点
    LNode *p = L,*q = NULL;
    while(p){
        q = p;
        p = p->next;
        free(q);
    }
}

```

```

    }
    return OK;
}

Status ClearList(LinkList L){
    //释放L头结点外的所有结点
    LNode *p = L->next, *q = NULL;
    while(p){
        q = p;
        p = p->next;
        free(q);
    }
    L->next = NULL;
    L->exp = 0;
    return OK;
}

Status InsertNode(LinkList L, int nexp, double ncoe){
    //插入新结点
    LNode *p = NULL, *q = NULL;
    p = (LNode*)malloc(sizeof(LNode));
    if(p == NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    p->exp = nexp;
    p->coef = ncoe;
    p->next = NULL; //完成新结点创建和输入
    q = L;
    while(q->next && q->next->exp > p->exp){
        q = q->next;
    }
    p->next = q->next;
    q->next = p;
    //向链表中按顺序插入新结点
    L->exp++;
    return OK;
}

Status DeleteNode(LinkList L, int location){
    //删除指定位置的结点
    LNode *p = NULL, *q = L;
    int i;
    for(i=1; i<location; i++){
        q = q->next; //q移动到需删除结点的前一位
    }
    p = q->next;
    q->next = p->next;
    free(p);
    L->exp--;
    return OK;
}

Status EditNode(LinkList L, int location, int nexp, double ncoe){
    LNode *p = NULL, *q = L;
    int i;
    for(i=1; i<location; i++){
        q = q->next; //q移动到需修改结点的前一位
    }
    p = q->next;

```

```

    q->next = p->next; //移除p
    p->exp = nexp;
    p->coef = ncoe;
    p->next = NULL; //给p赋新值
    q = L;
    while(q->next && q->next->exp > p->exp){
        q=q->next;
    }
    p->next = q->next;
    q->next = p; //插入p
    return OK;
}

```

```

LinkList DiffList(LinkList L){
    //计算L的一阶导数存入新链表，返回新链表的头指针
    LinkList Ln=(LinkList)malloc(sizeof(LNode));
    if(Ln==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    Ln->exp = 0;
    Ln->next = NULL;
    LNode *p = NULL, *q=Ln;
    LNode *p1 = L->next;
    while(p1 && p1->exp != 0){ //仅考察非常数项的导数
        p=(LNode*)malloc(sizeof(LNode));
        if(p==NULL){
            printf("内存错误!\n");
            exit(OVERFLOW);
        }
        p->exp = p1->exp - 1;
        p->coef = p1->coef * p1->exp;
        p->next = NULL;
        q->next = p;
        q = p;
        p1 = p1->next;
        Ln->exp++;
    }
    return Ln;
}

```

```

LinkList IntelList(LinkList L){
    //计算L的一阶积分（常数项为0）存入新链表，返回新链表的头指针
    LinkList Ln=(LinkList)malloc(sizeof(LNode));
    if(Ln==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    Ln->exp = 0;
    Ln->next = NULL;
    LNode *p = NULL, *q=Ln;
    LNode *p1 = L->next;
    while(p1){
        p=(LNode*)malloc(sizeof(LNode));
        if(p==NULL){
            printf("内存错误!\n");
            exit(OVERFLOW);
        }
    }
}

```



```

        p->exp = p1->exp + 1;
        p->coef = p1->coef / (p1->exp + 1);
        p->next = NULL;
        q->next = p;
        q = p;
        p1 = p1->next;
        Ln->exp++;
    }
    return Ln;
}

LinkList MultList(LinkList La, LinkList Lb){
    //创建一个新多项式，内容为两个输入的多项式相乘，返回结果的指针
    LinkList Lc=(LinkList)malloc(sizeof(LNode));
    LinkList Lsum=(LinkList)malloc(sizeof(LNode));
    LinkList temp = NULL;
    if(Lc==NULL || Lsum==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    Lc->exp=0;
    Lc->next = NULL;
    Lsum->exp=0;
    Lsum->next = NULL;
    LNode *p=NULL, *q=NULL;
    LNode *pa=La->next, *pb=Lb->next;
    while(pa){
        pb = Lb->next;
        q = Lc;
        while(pb){
            p=(LNode*)malloc(sizeof(LNode));
            if(p==NULL){
                printf("内存错误!\n");
                exit(OVERFLOW);
            }
            p->exp = pa->exp + pb->exp;
            p->coef = pa->coef * pb->coef;
            p->next = NULL;
            q->next = p;
            q = p;
            pb = pb->next;
            Lc->exp++;
        }
        temp = Lsum;
        Lsum = AddList(temp, Lc);
        DestroyList(temp);
        ClearList(Lc);
        pa = pa->next;
    }
    free(Lc);
    return Lsum;
}

DivResult DivList(LinkList La, LinkList Lb){
    //输入被除式与除式，以结构体形式输出商和余数的头指针
    DivResult res;
    LinkList Mult=NULL, temp = NULL;
    LNode *pa = La->next, *pb = Lb->next;

```

```

LNode *p = NULL,*q = NULL;
res.quotient = (LinkList)malloc(sizeof(LNode));
p = (LNode*)malloc(sizeof(LNode));
if(p==NULL){
    printf("内存错误!\n");
    exit(OVERFLOW);
}
p->coef = pa->coef / pb->coef;
p->exp = pa->exp - pb->exp;
p->next = NULL;
res.quotient->next = p;
res.quotient->exp = 1;
q = p;
Mult = MultList(res.quotient,Lb);
res.remainder = SubList(La,Mult);
DestroyList(Mult),Mult = NULL;
while(res.remainder->next != NULL && res.remainder->next->exp >= pb->exp){
    p = (LNode*)malloc(sizeof(LNode));
    if(p==NULL){
        printf("内存错误!\n");
        exit(OVERFLOW);
    }
    p->coef = res.remainder->next->coef / pb->coef;
    p->exp = res.remainder->next->exp - pb->exp;
    p->next = NULL;
    q->next = p;
    q = p;
    res.quotient->exp++;
    Mult = MultList(res.quotient,Lb);
    temp = res.remainder;
    res.remainder = SubList(La,Mult);
    DestroyList(temp);temp=NULL;
    DestroyList(Mult);Mult=NULL;
}
return res;
}

int menu(){
    int ope;
    do{
        printf("-----多项式计算器-----\n");
        printf("1、输入并创建多项式\n");
        printf("2、输出多项式\n");
        printf("3、求和\n");
        printf("4、求差\n");
        printf("5、求值\n");
        printf("6、销毁\n");
        printf("7、清空\n");
        printf("8、修改\n");
        printf("9、微分\n");
        printf("10、不定积分\n");
        printf("11、定积分\n");
        printf("12、乘法和乘方\n");
        printf("13、除法\n");
        /*printf("14. 多项式的四则运算\n");*/
        printf("0、退出程序\n");
        printf("-----\n");
        printf("请输入操作: ");
    }
}

```

```

scanf("%d",&o);
getchar();
system("cls");
if(o<0 || o>14){
    printf("输入有误, 请重新输入! \n");
    getchar();
    system("cls");
}
}while(o<0 || o>14);
return o;
}

int main(){
    LinkList p[NUM] = {NULL};
    //char formula[NUM*5] = {0};
    int i,j,m,n,l;
    double x,y;
    LinkList result=NULL,temp=NULL;
    DivResult divresult;
    char option;
    while(1){
        switch(menu()){
            case 0:
                for(i=0;i<NUM;i++){
                    if(p[i])
                        DestroyList(p[i]);
                }
                printf("感谢您的使用, 再见! \n");
                return OK;

            case 1:
                i=0;
                while(p[i]) i++;
                p[i]=(LinkList)malloc(sizeof(LNode));
                if(p[i]==NULL){
                    printf("内存错误!\n");
                    exit(OVERFLOW);
                }
                p[i]->exp=0;p[i]->next=NULL;
                if(InitList(p[i]))
                    printf("创建成功, 多项式编号为%d, 按回车键返回菜单\n",i+1);
                getchar();
                system("cls");
                break;

            case 2:
                printf("请输入多项式编号: ");
                scanf("%d",&i);
                getchar();
                if(p[i-1]){
                    if(PrintList(p[i-1])){
                        putchar('\n');
                        printf("输出完毕, 按回车键返回菜单\n");
                    }
                }
                else printf("该编号下无多项式! \n");
                getchar();
                system("cls");

```

```
break;
```

```
case 3:
```

```
printf("请输入需要相加的第一个多项式编号: ");
scanf("%d",&i);
getchar();
printf("请输入需要相加的第二个多项式编号: ");
scanf("%d",&j);
getchar();
if(p[i-1] && p[j-1]){
    result=AddList(p[i-1],p[j-1]);
    PrintList(result);
    putchar('\n');
    printf("要存储该结果吗? (Y/N): ");
    option = getchar();
    getchar();
    while(option != 'Y' && option != 'N'){
        printf("请重新输入:");
        option = getchar();
        getchar();
    }
    if(option == 'Y'){
        i=0;
        while(p[i]) i++;
        p[i]=result;
        printf("存储成功, 多项式编号为%d, 按回车键返回菜单\n",i+1);
    }
    if(option == 'N'){
        DestroyList(result);
        printf("计算完毕, 按回车键返回菜单\n",i+1);
    }
}
else printf("该编号下无多项式! \n");
getchar();
system("cls");
break;
```

```
case 4:
```

```
printf("请输入被减数多项式编号: ");
scanf("%d",&i);
getchar();
printf("请输入减数多项式编号: ");
scanf("%d",&j);
getchar();
if(p[i-1] && p[j-1]){
    result=SubList(p[i-1],p[j-1]);
    PrintList(result);
    putchar('\n');
    printf("要存储该结果吗? (Y/N): ");
    option = getchar();
    getchar();
    while(option != 'Y' && option != 'N'){
        printf("请重新输入:");
        option = getchar();
        getchar();
    }
    if(option == 'Y'){
        i=0;
```

```

        while(p[i]) i++;
        p[i]=result;
        printf("存储成功, 多项式编号为%d, 按回车键返回菜单\n",i+1);
    }
    if(option == 'N'){
        DestroyList(result);
        printf("计算完毕, 按回车键返回菜单\n",i+1);
    }
}
else printf("该编号下无多项式! \n");
getchar();
system("cls");
break;

case 5:
    printf("请输入多项式编号: ");
    scanf("%d",&i);
    getchar();
    if(p[i-1]){
        printf("请输入x的值: ");
        scanf("%lf",&x);
        getchar();
        x = EvaluateList(p[i-1],x);
        printf("代入计算结果为 %.3lf, 按回车键返回菜单\n",x);
    }
    else printf("该编号下无多项式! \n");
    getchar();
    system("cls");
    break;

case 6:
    printf("请输入多项式编号: ");
    scanf("%d",&i);
    getchar();
    if(p[i-1]){
        if(DestroyList(p[i-1]))
            p[i-1]=NULL;
        printf("销毁完毕, 按回车键返回菜单\n");
    }
    else printf("该编号下无多项式! \n");
    getchar();
    system("cls");
    break;

case 7:
    printf("请输入多项式编号: ");
    scanf("%d",&i);
    getchar();
    if(p[i-1])
        if(ClearList(p[i-1]))
            printf("清空完毕, 按回车键返回菜单\n");
    else printf("该编号下无多项式! \n");
    getchar();
    system("cls");
    break;

case 8:
    printf("请输入多项式编号: ");

```

```

scanf("%d",&i);
getchar();
if(p[i-1]){
    do{
        printf("请选择操作: \n");
        printf("1、插入新的结点\n");
        printf("2、删除已有结点\n");
        printf("3、修改已有结点的系数和指数\n");
        printf("0、退出\n");
        scanf("%d",&n);
        getchar();
        system("cls");
        if(n<0 || n>3){
            printf("输入有误, 请重新输入! \n");
            getchar();
            system("cls");
        }
    }while(n<0 || n>3);
    switch(n){
        case 1:
            do{
                printf("请输入新结点的指数 (输入-1取消): ");
                scanf("%d",&j);
                getchar();
                while(j < -1){
                    printf("指数不合法, 请重新输入:(输入-1结束输入):");
                    scanf("%d",&j);
                    getchar();
                }
                l = Locate(p[i-1],j);
                if(l) printf("该指数项已存在, 请重新输入! \n");
            }while(l);
            if(j == -1) break;
            printf("请输入新结点的系数: ");
            scanf("%lf",&x);
            getchar();
            if(x != 0)
                if(InsertNode(p[i-1],j,x)){
                    printf("插入成功! 当前");
                    PrintList(p[i-1]);
                    putchar('\n');
                }
            break;

        case 2:
            do{
                printf("请输入删除结点的指数 (输入-1取消): ");
                scanf("%d",&j);
                getchar();
                l = Locate(p[i-1],j);
                if(!l && j >= 0) printf("未找到该项, 请重新输入! \n");
            }while(!l && j >= 0);
            if(j == -1) break;
            if(DeleteNode(p[i-1],l)){
                printf("删除成功! 当前");
                PrintList(p[i-1]);
                putchar('\n');
            }
    }
}

```

```

        break;

    case 3:
        do{
            printf("请输入需修改结点的指数（输入-1取消）：");
            scanf("%d",&j);
            getchar();
            l = Locate(p[i-1],j);
            if(!l && j >= 0) printf("未找到该项，请重新输入！\n");
        }while(!l && j >= 0);
        if(j == -1) break;
        do{
            printf("请输入新指数：");
            scanf("%d",&m);
            getchar();
            n = Locate(p[i-1],m);
            if(m != j && n) printf("已存在该指数项，请重新输入！\n");
        }while(m != j && n);
        printf("请输入新系数：");
        scanf("%lf",&x);
        getchar();
        if(x != 0){
            if(EditNode(p[i-1],l,m,x)){
                printf("修改成功！当前");
                PrintList(p[i-1]);
                putchar('\n');
            }
        }
        else if(DeleteNode(p[i-1],l)){
            printf("修改成功！当前");
            PrintList(p[i-1]);
            putchar('\n');
        }
        break;
    case 0: break;
}

}

else printf("该编号下无多项式！\n");
printf("按回车键继续\n");
getchar();
system("cls");
break;

case 9:
    printf("请输入需要微分的多项式编号：");
    scanf("%d",&i);
    getchar();
    if(p[i-1]){
        printf("请输入需求的微分阶数：");
        scanf("%d",&n);
        getchar();
        while(n <= 0){
            printf("需求的微分阶数输入有误，请重新输入：");
            scanf("%d",&n);
            getchar();
        }
        result=DiffList(p[i-1]);
        for(j=1;j<n;j++){

```

```

        temp = result;
        result = DiffList(temp);
        DestroyList(temp);
    }
    temp = NULL;
    PrintList(result);
    putchar('\n');
    printf("要存储该结果吗? (Y/N): ");
    option = getchar();
    getchar();
    while(option != 'Y' && option != 'N'){
        printf("请重新输入:");
        option = getchar();
        getchar();
    }
    if(option == 'Y'){
        i=0;
        while(p[i]) i++;
        p[i]=result;
        printf("存储成功, 多项式编号为%d, 按回车键返回菜单\n",i+1);
    }
    if(option == 'N'){
        DestroyList(result);
        printf("计算完毕, 按回车键返回菜单\n",i+1);
    }
}
else printf("该编号下无多项式! \n");
getchar();
system("cls");
break;

```

case 10:

```

printf("请输入需要求不定积分的多项式编号: ");
scanf("%d",&i);
getchar();
if(p[i-1]){
    result=IntelList(p[i-1]);
    PrintList(result);
    printf("+C (C为任意常数) \n");
    printf("要存储该结果吗? (Y/N): ");
    option = getchar();
    getchar();
    while(option != 'Y' && option != 'N'){
        printf("请重新输入:");
        option = getchar();
        getchar();
    }
    if(option == 'Y'){
        i=0;
        while(p[i]) i++;
        p[i]=result;
        printf("存储成功, 多项式编号为%d, 按回车键返回菜单\n",i+1);
    }
    if(option == 'N'){
        DestroyList(result);
        printf("计算完毕, 按回车键返回菜单\n",i+1);
    }
}
}

```



```

else printf("该编号下无多项式! \n");
getchar();
system("cls");
break;

case 11:
printf("请输入需要求定积分的多项式编号: ");
scanf("%d",&i);
getchar();
if(p[i-1]){
printf("请输入定积分起点: ");
scanf("%lf",&x);
getchar();
printf("请输入定积分终点: ");
scanf("%lf",&y);
getchar();
result=IntelList(p[i-1]);
printf("定积分结果为: %.3lf \n",EvaluateList(result,y)-
EvaluateList(result,x));
printf("计算完毕, 按回车键返回菜单\n",i+1);
}
else printf("该编号下无多项式! \n");
getchar();
system("cls");
break;

case 12:
do{
printf("1. 多项式相乘\n");
printf("2. 多项式乘方\n");
printf("请输入操作: ");
scanf("%d",&n);
getchar();
system("cls");
if(n != 1 && n != 2){
printf("输入有误, 请重新输入! \n");
getchar();
system("cls");
}
}while(n != 1 && n != 2);
if(n == 1){
printf("请输入需要相乘的第一个多项式编号: ");
scanf("%d",&i);
getchar();
printf("请输入需要相乘的第二个多项式编号: ");
scanf("%d",&j);
getchar();
if(p[i-1] && p[j-1]){
result=MultList(p[i-1],p[j-1]);
PrintList(result);
putchar('\n');
printf("要存储该结果吗? (Y/N): ");
option = getchar();
getchar();
while(option != 'Y' && option != 'N'){
printf("请重新输入:");
option = getchar();
getchar();
}
}
}

```

```

    }
    if(option == 'Y'){
        i=0;
        while(p[i]) i++;
        p[i]=result;
        printf("存储成功, 多项式编号为%d, 按回车键返回菜单\n", i+1);
    }
    if(option == 'N'){
        DestroyList(result);
        printf("计算完毕, 按回车键返回菜单\n", i+1);
    }
}
else printf("该编号下无多项式! \n");
}
if(n == 2){
    printf("请输入需要乘方的多项式编号: ");
    scanf("%d",&i);
    getchar();
    if(p[i-1]){
        printf("请输入乘方次数: ");
        scanf("%d",&n);
        getchar();
        while(n <= 1){
            printf("乘方次数输入有误, 请重新输入: ");
            scanf("%d",&n);
            getchar();
        }
        result = MultList(p[i-1],p[i-1]);
        for(j=2;j<n;j++){
            temp = result;
            result = MultList(temp,p[i-1]);
            DestroyList(temp);
        }
        temp = NULL;
        PrintList(result);
        putchar('\n');
        printf("要存储该结果吗? (Y/N): ");
        option = getchar();
        getchar();
        while(option != 'Y' && option != 'N'){
            printf("请重新输入:");
            option = getchar();
            getchar();
        }
        if(option == 'Y'){
            i=0;
            while(p[i]) i++;
            p[i]=result;
            printf("存储成功, 多项式编号为%d, 按回车键返回菜单\n", i+1);
        }
        if(option == 'N'){
            DestroyList(result);
            printf("计算完毕, 按回车键返回菜单\n", i+1);
        }
    }
    else printf("该编号下无多项式! \n");
}
getchar();

```

```

        system("cls");
        break;

    case 13:
        printf("请输入被除数多项式编号: ");
        scanf("%d",&i);
        getchar();
        printf("请输入除数多项式编号: ");
        scanf("%d",&j);
        getchar();
        if(p[i-1] && p[j-1]){
            if(p[i-1]->next == NULL){
                printf("被除数为0, 结果为0\n");
            }
            else{
                if(p[j-1]->next == NULL){
                    printf("除数为0\n");
                }
                else{
                    if(p[i-1]->next->exp < p[j-1]->next->exp){
                        printf("被除数最高次数小于除数最高次数\n");
                    }
                    else{
                        divresult = DivList(p[i-1],p[j-1]);
                        printf("商");
                        PrintList(divresult.quotient);
                        putchar('\n');
                        printf("余数");
                        PrintList(divresult.remainder);
                        putchar('\n');
                        printf("要存储商吗? (Y/N): ");
                        option = getchar();
                        getchar();
                        while(option != 'Y' && option != 'N'){
                            printf("请重新输入:");
                            option = getchar();
                            getchar();
                        }
                        if(option == 'Y'){
                            i=0;
                            while(p[i]) i++;
                            p[i] = divresult.quotient;
                            printf("存储成功, 多项式编号为%d\n",i+1);
                        }
                        if(option == 'N'){
                            DestroyList(divresult.quotient);
                        }
                        printf("要存储余数吗? (Y/N): ");
                        option = getchar();
                        getchar();
                        while(option != 'Y' && option != 'N'){
                            printf("请重新输入:");
                            option = getchar();
                            getchar();
                        }
                        if(option == 'Y'){
                            i=0;
                            while(p[i]) i++;

```

```

        p[i] = divresult.remainder;
        printf("存储成功, 多项式编号为%d, 按回车键返回菜单\n", i+1);
    }
    if(option == 'N'){
        DestroyList(divresult.remainder);
        printf("计算完毕, 按回车键返回菜单\n", i+1);
    }
}
}
}
}
else printf("该编号下无多项式! \n");
getchar();
system("cls");
break;
}
}
}

```

栈与队列的应用

简单的行文本编辑器

```

#include <stdio.h>
#include <stdlib.h>
#define OK 1
#define ERROR 0
#define OVERFLOW -2
#define MAXSIZE 500
typedef int Status;

typedef struct SqStack{
    char *base;
    char *top;
    int stacksize;
}SqStack;

int strcmp(char *A, char *B){
    char *pa = A, *pb = B;
    while(*pa && *pb){
        if(*pa > *pb) return 1;
        if(*pa < *pb) return -1;
        pa++; pb++;
    }
    if(*pa && !*pb) return 1;
    if(!*pa && *pb) return -1;
    if(!*pa && !*pb) return 0;
} //字符串比较函数

int JudgeOpe(char *ope){
    if(strcmp(ope, "Get") == 0) return 1;
    else if(strcmp(ope, "Prev") == 0) return 2;
    else if(strcmp(ope, "Next") == 0) return 3;
}

```

```

else if(strcmp(ope,"Move") == 0) return 4;
else if(strcmp(ope,"Insert") == 0) return 5;
else if(strcmp(ope,"Delete") == 0) return 6;
else if(strcmp(ope,"Rotate") == 0) return 7;
else return 0;
} //判定操作内容，转换为编号

Status Move(SqStack *PText, SqStack *NText, int num){
    int i;
    int ssp=PText->stacksize, ssn=NText->stacksize;
    if(num < ssp){
        //需移动位置在当前光标位置之前
        //将PText中的ssp-num个元素出栈，放入NText
        for(i=0; i<ssp-num; i++){
            *NText->top = *(PText->top-1);
            *(PText->top-1) = 0;
            NText->top--; PText->top--;
            NText->stacksize++;
            PText->stacksize--;
        }
    }
    if(num > ssp){
        //需移动位置在当前光标位置之后
        //将NText中的num-ssp个元素出栈，放入PText
        for(i=0; i<num-ssp; i++){
            *PText->top = *(NText->top+1);
            *(NText->top+1) = 0;
            NText->top++; PText->top++;
            NText->stacksize--;
            PText->stacksize++;
        }
    }
    return OK;
}

Status Prev(SqStack *PText, SqStack *NText){
    //将PText栈顶元素出栈，放入NText
    *NText->top = *(PText->top-1);
    NText->top--; PText->top--;
    NText->stacksize++;
    PText->stacksize--;
    return OK;
}

Status Next(SqStack *PText, SqStack *NText){
    //将NText栈顶元素出栈，放入PText
    *PText->top = *(NText->top+1);
    NText->top++; PText->top++;
    NText->stacksize--;
    PText->stacksize++;
    return OK;
}

Status Insert(SqStack *PText, SqStack *NText, FILE *fp, int num){
    //从fp开始读入字符存入PText，再将光标移动至原处
    int i;
    int start = PText->stacksize; //记录原光标位置
    fgetc(fp);

```

```

        for(i=0;i<num;i++){
            *PText->top = fgetc(fp);
            PText->top++;
            PText->stacksize++;
        }
        Move(PText,NText,start);
        return OK;
    }

Status Get(SqStack *PText,SqStack *NText,FILE *nfp){
    //输出当前内容至文件和显示器
    int i;
    for(i=0;i<PText->stacksize;i++){
        fputc(*(PText->base+i),nfp);
        putchar(*(PText->base+i));
    }
    for(i=1;i<=NText->stacksize;i++){
        fputc(*(NText->top+i),nfp);
        putchar(*(NText->top+i));
    }
    putchar('\n');
    return OK;
}

Status Delete(SqStack *NText,int num){
    //只需将NText中num个元素出栈即可
    int i;
    for(i=0;i<num;i++){
        *(NText->top+1) = 0;
        NText->top++;
        NText->stacksize--;
    }
    return OK;
}

Status Rotate(SqStack *PText,SqStack *NText,int num){
    //将NText栈顶num个元素倒序放入栈内
    char *temp = NULL;
    temp = (char*)malloc(num*sizeof(char));
    int i;
    for(i=0;i<num;i++){
        temp[i] = *(NText->top+1);
        NText->top++;
    }
    for(i=0;i<num;i++){
        *NText->top = temp[i];
        NText->top--;
    }
    free(temp);
    return OK;
}

Status main(){
    FILE *fp,*nfp;
    int Openum,num;
    int i,j;
    char ope[7];
    char v[MAXSIZE] = {0}; //构建顺序存储空间

```

```

SqStack *PText,*NText;
PText = (SqStack*)malloc(sizeof(SqStack));
NText = (SqStack*)malloc(sizeof(SqStack));
//栈PText存放光标前内容，栈NText存放光标后内容
//栈PText和NText共用数组空间V
PText->base = V;
PText->top = V;
PText->stacksize = 0;
NText->base = V + MAXSIZE - 1;
NText->top = V + MAXSIZE - 1;
NText->stacksize = 0;
//初始化栈底与栈顶指针
fp = fopen("F:/Input/input.txt","r");
if(fp == NULL){
    printf("打开文件失败! \n");
    exit(ERROR);
}
fscanf(fp,"%d",&OpenNum);
if(OpenNum <= 0 || OpenNum > 50000){
    printf("指令数超出上限! \n");
    return ERROR;
}
for(i = 0;i < OpenNum;i++){
    fscanf(fp,"%s",ope);
    switch(JudgeOpe(ope)){
        case 1:
            nfp = fopen("F:/Input/output.txt","w");
            Get(PText,NText,nfp);
            break;
        case 2:
            Prev(PText,NText);
            break;
        case 3:
            Next(PText,NText);
            break;
        case 4:
            fscanf(fp,"%d",&num);
            Move(PText,NText,num);
            break;
        case 5:
            fscanf(fp,"%d",&num);
            Insert(PText,NText,fp,num);
            break;
        case 6:
            fscanf(fp,"%d",&num);
            Delete(NText,num);
            break;
        case 7:
            fscanf(fp,"%d",&num);
            Rotate(PText,NText,num);
            break;
        default:
            printf("非有效指令! \n");
            return ERROR;
    }
}
return OK;
}

```

括号配对检验

```
#include<stdio.h>
#include<stdlib.h>
#define OK 1
#define ERROR 0
#define TRUE 1
#define FALSE 0
#define OVERFLOW -2
#define MAXSIZE 1000
#define STACKMAXSIZE 100
typedef int Status;

typedef struct{
    char *base;
    char *top;
}Stack, *SqStack;

Status InitStack(SqStack S){
    //构造空顺序栈S
    S->base = (char*)malloc(sizeof(char)*STACKMAXSIZE);
    if(!S->base){
        printf("存储空间分配失败! \n");
        exit(OVERFLOW);
    }
    S->top = S->base;
    return OK;
}

Status Push(SqStack S, char c){
    //c入栈
    *S->top = c;
    S->top++;
    return OK;
}

char GetTop(SqStack S){
    //取栈顶元素
    return *(S->top-1);
}

char Pop(SqStack S){
    //栈顶元素出栈
    char c;
    c = *(S->top-1);
    S->top--;
    return c;
}

Status BracketCmp(char a, char b){
    //判定括号a是否比括号b优先级低
    int la, lb;
    if(a == '(') la = 1;
    if(a == '[') la = 2;
    if(a == '{') la = 3;
    if(b == '(') lb = 1;
```



```

    if(b == '[') lb = 2;
    if(b == '{') lb = 3;
    if(la < lb) return TRUE;
    else return FALSE;
}

Status BracketPairing(char a,char b){
    //判定右括号a是否与左括号b配对
    int la,lb;
    if(a == ')') la = 1;
    if(a == ']') la = 2;
    if(a == '}') la = 3;
    if(b == '(') lb = 1;
    if(b == '[') lb = 2;
    if(b == '{') lb = 3;
    if(la == lb) return TRUE;
    else return FALSE;
}

Status CheckBracket(char *text,SqStack S){
    //检查括号是否合法，若合法返回TRUE，否则FALSE
    char *pc = text;
    while(*pc != '@'){
        if(*pc == '(' || *pc == '[' || *pc == '{'){
            //字符为左括号
            if(S->base == S->top || BracketCmp(*pc,GetTop(S))){
                //若栈空，或该括号比栈顶括号优先级低，入栈
                Push(S,*pc);
            }
            else return FALSE;
            //若该括号不比栈顶括号优先级低，则错误
        }
        if(*pc == ')' || *pc == ']' || *pc == '}'){
            //字符为右括号
            if(S->base != S->top && BracketPairing(*pc,GetTop(S))){
                //若栈非空，且括号与栈顶左括号配对，出栈
                Pop(S);
            }
            else return FALSE;
            //若该括号不与栈顶括号配对或栈空，则错误
        }
        pc++;
    }
    if(S->base != S->top)
        //若此时栈不为空，则错误
        return FALSE;
    //完成所有括号入栈与出栈，括号合法
    return TRUE;
}

int main(){
    int i,num;
    char text[MAXSIZE] = {'\0'};
    SqStack S;
    int *result = NULL;//存放结果
    S = (SqStack)malloc(sizeof(Stack));
    InitStack(S);
    scanf("%d",&num);

```

```

result = (int*)malloc(num*sizeof(int));
if(!result){
    printf("存储空间分配失败! \n");
    exit(OVERFLOW);
}
for(i = 0;i < num;i++){
    scanf("%s",text);
    result[i] = CheckBracket(text,S);
}
for(i = 0;i < num;i++){
    if(result[i])
        printf("YES\n");
    else
        printf("NO\n");
}
return OK;
}

```

银行业务模拟

```

#include<stdio.h>
#include<stdlib.h>
#define OK 1
#define ERROR 0
#define OVERFLOW -2
typedef int Status;

typedef struct Customer{
    int Money;
    int ArriveTime;
    int LeaveTime;
    struct Customer *next;
}Customer;

typedef struct Event{
    int OccurTime; //事件的发生时间
    int EventType; //1表示进入, 0表示离开
    struct Event *next;
}Event,*LinkList;

typedef struct QNode{
    Customer *ctm;
    struct QNode *next;
}QNode;

typedef struct Queue{
    QNode *front;
    QNode *rear;
}Queue,*LinkQueue;

int N,total,close_time,average_time;
int t=0;

Status InputCustomer(Customer *C,int N){
    //将客户信息输入带头结点链表
    int i=0;
    int m,at;

```

```

Customer *p = NULL,*q = C;
for(i=0;i<N;i++){
    p=(Customer*)malloc(sizeof(Customer));
    if(p==NULL){
        printf("内存错误! \n");
        exit(OVERFLOW);
    }
    scanf("%d %d",&m,&at);
    p->Money = m;
    p->ArriveTime = at;
    p->LeaveTime = 0;
    p->next = NULL;
    q->next = p;
    q = p;
}
return OK;
}

Status EnQueue(LinkQueue Q, Customer *c){
    //将客户c插入队尾
    QNode *p = NULL;
    p = (QNode*)malloc(sizeof(QNode));
    p->ctm = c;
    p->next = NULL;
    Q->rear->next = p;
    Q->rear = p;
    return OK;
}

Status EnList(LinkList events, int time, int EventType){
    //按时间顺序插入新事件
    Event *p = events;
    Event *q = NULL;
    q = (Event*)malloc(sizeof(Event));
    q->next = NULL;
    q->EventType = EventType;
    q->OccurTime = time;
    while(p->next && p->next->OccurTime <= time){
        p = p->next;
    }
    q->next = p->next;
    p->next = q;
    return OK;
}

int QueueLength(LinkQueue Q){
    //检查队列Q长度
    int n=0;
    QNode *p = Q->front->next;
    while(p){
        n++;
        p = p->next;
    }
    return n;
}

Customer *DeFrontQueue(LinkQueue Q){
    //删去Q队头，返回队头客户指针

```

```

    Customer *c;
    QNode *p = Q->front->next;
    c = p->ctm;
    Q->front->next = p->next;
    free(p);
    if(Q->front->next == NULL)
        Q->rear = Q->front;
    return c;
}

Event DelFirstEvent(LinkList events){
    //删除事件列表第一个结点，返回其事件
    Event *p = events->next;
    Event e;
    e.EventType = p->EventType;
    e.OccurTime = p->OccurTime;
    e.next = NULL;
    events->next = p->next;
    free(p);
    return e;
}

Status CustomerArrived(LinkList events, LinkQueue Q1, Customer *nctm){
    //客户到达事件
    //需要执行：将当前客户插入队列1；
    //将下一个客户到达事件插入事件表
    //若队列1只剩一个，将该客户离开事件插入事件表
    EnQueue(Q1, nctm);
    //printf("Customer Arrived!");
    if(nctm->next != NULL){
        EnList(events, nctm->next->ArriveTime, 1);
    }
    //最后一个客户不需要引入到达事件
    if(QueueLength(Q1) == 1 && nctm->ArriveTime + average_time <= close_time){
        EnList(events, nctm->ArriveTime + average_time, 0);
    }
    return OK;
}

int checkQ2(LinkQueue Q2, int BFTotal, int time){
    //检查Q2，返回办理用户的个数
    Customer *c;
    int num = QueueLength(Q2);
    int i=0, n=0;
    //记录Q2长度
    while(total >= BFTotal && i < num){
        //循环结束条件为当前存款数低于上一次存款前存款数
        //或检查完毕Q2
        c = DelFrontQueue(Q2);
        if(total + c->Money >= 0){
            //存款足够，处理当前客户
            if(time + average_time*n <= close_time){
                total += c->Money;
                n++;
                c->LeaveTime = time + average_time*n;
            }
        }
        else{

```

```

        //存款不足，把当前客户放到队尾
        EnQueue(Q2,c);
    }
    i++;
}
return n;
}

Status DestroyQueue(LinkQueue Q){
    //删除队列
    QNode *p = Q->front,*q;
    while(p){
        q = p->next;
        free(p);
        p = q;
    }
    return OK;
}

Status DestroyCustomers(Customer *c){
    //释放顾客链表
    Customer *p = c,*q;
    while(p){
        q = p->next;
        free(p);
        p = q;
    }
    return OK;
}

Status CustomerDeparture(LinkList events,Event pe,LinkQueue Q1,LinkQueue Q2){
    //客户离开事件
    //需要执行：移除队列1队头客户
    //判断该用户是否去队列2
    //判断该用户是否是存款用户
    //若队列1不空，队列1中下一个客户的离开事件插入事件表
    Customer *c;
    c = DelFrontQueue(Q1);
    int n=0;
    if(total + c->Money >= 0){
        //用户不需去队列2，修改存款，计算离开时间
        total += c->Money;
        c->LeaveTime = pe.OccurTime;
    }
    else{
        //用户需要去队列2
        EnQueue(Q2,c);
    }
    if(c->Money > 0){
        //若为存款用户，检查队列2
        n = CheckQ2(Q2,total - c->Money,pe.OccurTime);
        //n标记Q2中办理手续的用户数
    }
    if(QueueLength(Q1) > 0 && pe.OccurTime + average_time + n*average_time <=
close_time){
        //队列1不空
        EnList(events,pe.OccurTime + average_time + n*average_time ,0);
        //下一个离开事件发生的时间为：

```

```

        //当前时间+下一个客户接待时间+处理Q2所需时间
    }
    return OK;
}

int main(){
    int TotalTime = 0;
    scanf("%d %d %d %d",&N,&total,&close_time,&average_time);
    Customer *customers,*nctm = NULL;
    customers = (Customer*)malloc(sizeof(Customer));
    customers->next = NULL;
    InputCustomer(customers,N);
    nctm = customers->next;
    //生成并读取客户信息链表
    LinkList events;
    Event pe;
    events = (LinkList)malloc(sizeof(Event));
    events->next = NULL;
    EnList(events,nctm->ArriveTime,1);
    //创建事件列表,将第一个客户到达事件插入链表
    LinkQueue Q1,Q2;
    Q1 = (LinkQueue)malloc(sizeof(Queue));
    Q2 = (LinkQueue)malloc(sizeof(Queue));
    Q1->front = (QNode*)malloc(sizeof(QNode));
    Q2->front = (QNode*)malloc(sizeof(QNode));
    Q1->front->next = NULL;
    Q2->front->next = NULL;
    Q1->rear = Q1->front;
    Q2->rear = Q2->front;
    //创建含头结点队列Q1,Q2并完成初始化
    while(events->next != NULL){
        pe = DelFirstEvent(events);
        if(pe.EventType == 1){
            //为到达事件
            CustomerArrived(events,Q1,nctm);
            nctm = nctm->next;
        }
        if(pe.EventType == 0){
            //为离开事件
            CustomerDeparture(events,pe,Q1,Q2);
        }
    }
    //处理完所有事件
    nctm = customers->next;
    while(nctm){
        if(nctm->LeaveTime != 0){
            printf("%d\n",nctm->LeaveTime - nctm->ArriveTime - average_time);
            TotalTime += nctm->LeaveTime - nctm->ArriveTime - average_time;
        }
        else{
            //leavetime为0说明一直等待到银行关门
            printf("%d\n",close_time - nctm->ArriveTime);
            TotalTime += close_time - nctm->ArriveTime;
        }
        nctm = nctm->next;
    }
    printf("%d",TotalTime/N);
    free(events);
}

```

```

    DestroyQueue(Q1);
    DestroyQueue(Q2);
    free(Q1);free(Q2);
    DestroyCustomers(customers);//清理内存
    return OK;
}

```

二叉树的应用（哈夫曼树）

```

//需要完成的任务：
//压缩文件：
//  读入文件，统计每种字节出现的次数（256种）
//  根据字节出现的次数创建Huffman树
//  由Huffman树得到各字节的Huffman编码
//  读入文件，按字节将对应的编码写入.huff文件
//  将压缩前文件类型、各字节出现的次数信息写入.huff文件
//解压缩文件：
//  读入.huff文件，根据文件开头的信息获得Huffman树
//  根据Huffman编码，将对应的字节写入文件
//-----
//.huff文件的结构为：
//char FileType[FILE_TYPE_LENGTH] 源文件类型（,xxx）
//int FileSize 源文件大小（用于解压时确定是否解码结束）
//int count[256] 各字节按顺序的出现次数
//Huffman码比特流

#include<stdio.h>
#include<stdlib.h>
#include<windows.h>
#include<string.h>
#define OK 1
#define TRUE 1
#define FALSE 0
#define OVERFLOW -2
#define FILE_TYPE_LENGTH 6
typedef int Status;

typedef struct Info{
    int count; //无符号字符出现的次数
    char *HCode; //字符对应的Huffman编码
}Info; //文件内无符号字符的信息

typedef struct HTNode{
    unsigned char data;
    int weight;
    int lchild,rchild;
    int parent;
}HTNode,*HuffmanTree;

int menu(){
    //输出菜单，返回操作
    int operation;
    printf("Huffman码压缩/解压程序\n");
    printf("-----\n");
    printf("1.压缩文件\n");
    printf("2.解压文件\n");
}

```

```

    printf("0.退出\n");
    printf("-----\n");
    printf("请输入操作: ");
    scanf("%d",&operation);
    return operation;
}

Status CountByte(FILE *OriginFile,Info *FileByteInfo,int &types,int &FileSize){
    //统计字符出现次数
    unsigned char ReadBuff[1024];
    int ReadSize;
    int i;
    //每次读取1KB数据
    while(1){
        ReadSize = fread(ReadBuff,1,1024,OriginFile);
        if(ReadSize == 0){
            //读取完毕
            break;
        }
        FileSize += ReadSize;
        for(i=0;i<ReadSize;i++){
            if(FileByteInfo[ReadBuff[i]].count == 0)
                types++;
            FileByteInfo[ReadBuff[i]].count++;
        }
    }
    return OK;
}

Status FindSmallest(HuffmanTree T,int i,int &s1,int &s2){
    //找到数组中权重最小且未被连接过的两个结点
    int j = 1;
    while(T[j].parent) j++;
    s1 = j;
    for(j++;j < i;j++){
        if(T[j].weight < T[s1].weight && T[j].parent == 0) s1=j;
    }
    j=1;
    while(T[j].parent || j == s1) j++;
    s2 = j;
    for(j++;j < i;j++){
        if(T[j].weight < T[s2].weight && j!=s1 && T[j].parent == 0) s2=j;
    }
    return OK;
}

Status InitHuffmanTree(HuffmanTree &T,Info *FileByteInfo,int types){
    //建立Huffman树（顺序存储类型）
    T = (HTNode*)malloc(sizeof(HTNode)*types*2);
    int i=1,j=0;
    int s1,s2;
    for(i=1;i<=2*types-1;i++){
        T[i].lchild = 0; T[i].rchild = 0;
        T[i].parent = 0;
    }
    i=1;
    for(j=0;j<256;j++){
        if(FileByteInfo[j].count){

```



```

        T[i].data = j;
        T[i].weight = FileByteInfo[j].count;
        i++;
    }
} //完成每个结点的输入
for(i=types+1; i <= 2*types-1; i++){
    FindSmallest(T, i, s1, s2);
    //找到1到i-1中权重最小且未被连接过的两个结点
    T[i].lchild = s1;
    T[i].rchild = s2;
    T[i].weight = T[s1].weight + T[s2].weight;
    T[s1].parent = i; T[s2].parent = i;
    //完成连接
} //完成Huffman树的创建，最后一个节点为根
return OK;
}

Status HuffmanCoding(HuffmanTree T, Info *FileByteInfo, int types){
    //根据Huffman树得到各字符的Huffman编码
    char *cd = (char*)malloc(sizeof(char)*types);
    //构建工作区cd
    cd[types-1] = '\0';
    int i;
    int c, f, start;
    for(i=1; i<=types; i++){
        start = types-1;
        c = i; f = T[i].parent;
        while(f){
            if(c == T[f].lchild){
                cd[--start] = '0';
            }
            else{
                cd[--start] = '1';
            }
            c = f;
            f = T[f].parent;
        }
        FileByteInfo[T[i].data].HCode = (char*)malloc(sizeof(char)*(types-
start));
        strcpy(FileByteInfo[T[i].data].HCode, &cd[start]);
    }
    return OK;
}

Status ReNaming(char *FileName, char *FileType, int mode){
    //更改文件后缀
    //mode 1为更改为.huff后缀
    //mode 2为将.huff后缀改回原后缀
    char *p = FileName, *q, *pt = FileType;
    while(*p != '.') p++;
    q = p;
    if(mode == 1){
        while(*q){
            *pt = *q;
            pt++;
            *q = '\0';
            q++;
        }
    }
}

```

```

        //清空原有后缀，将原有后缀保存到FileType中
        strcpy(p, ".huff");
    }
    if(mode == 2){
        while(*q){
            *q = '\0';
            q++;
        }
        //清空原有后缀
        strcpy(p, FileType);
    }
    return OK;
}

Status WriteInfo(FILE *NewFile, Info *FileByteInfo){
    //将各字节的出现次数写入文件
    int i;
    for(i=0; i<256; i++){
        fwrite(&(FileByteInfo[i].count), sizeof(int), 1, NewFile);
    }
    return OK;
}

Status WriteHuffmanCode(FILE *OriginFile, FILE *NewFile, Info *FileByteInfo){
    //按源文件写入对应的Huffman编码
    unsigned char ReadBuff[1024];
    int ReadSize;
    unsigned char ch = 0;
    int BitCount = 0;
    char *code = NULL;
    int i;
    //每次读取1KB数据
    while(1){
        ReadSize = fread(ReadBuff, 1, 1024, OriginFile);
        if(ReadSize == 0){
            //读取完毕
            break;
        }
        for(i=0; i<ReadSize; i++){
            code = FileByteInfo[ReadBuff[i]].HCode;
            while(*code){
                ch = ch << 1;
                if(*code == '1')
                    ch = ch | 1;
                BitCount++;
                if(BitCount == 8){
                    fputc(ch, NewFile);
                    BitCount = 0;
                }
                code++;
            }
        }
    }
    if(BitCount != 0){
        //剩余不足1字节的编码未写入
        ch = ch << (8-BitCount);
        fwrite(&ch, 1, 1, NewFile);
    }
}

```

```

        return OK;
    }

Status HuffmanCompress(Info *FileByteInfo){
    //压缩过程
    char FileName[100] = {0};
    char FileType[FILE_TYPE_LENGTH] = {0};
    int types = 0, FileSize = 0;
    HuffmanTree T = NULL;
    FILE *OriginFile, *NewFile;
    printf("请输入文件路径:\n");
    scanf("%s", FileName);
    if(!(OriginFile = fopen(FileName, "rb"))){
        printf("打开文件错误! \n");
        exit(ERROR);
    }
    //打开源文件
    CountByte(OriginFile, FileByteInfo, types, FileSize);
    //完成字符数统计
    InitHuffmanTree(T, FileByteInfo, types);
    //创建Huffman树
    HuffmanCoding(T, FileByteInfo, types);
    //完成Huffman编码
    rewind(OriginFile);
    ReNaming(FileName, FileType, 1);
    if(!(NewFile = fopen(FileName, "wb"))){
        printf("打开文件错误! \n");
        exit(ERROR);
    }
    //创建.huff文件
    fwrite(FileType, sizeof(char), FILE_TYPE_LENGTH, NewFile);
    //写入源文件类型
    fwrite(&FileSize, sizeof(int), 1, NewFile);
    //写入源文件大小
    WriteInfo(NewFile, FileByteInfo);
    //写入字符数信息
    WriteHuffmanCode(OriginFile, NewFile, FileByteInfo);
    //写入Huffman编码
    fclose(OriginFile);
    fclose(NewFile);
    printf("压缩完成! 压缩文件存储路径为: %s\n", FileName);
    return OK;
}

Status GetInfo(FILE *OriginFile, Info *FileByteInfo, int &types){
    //从文件中读入字符数信息
    int i;
    for(i=0; i<256; i++){
        fread(&(FileByteInfo[i].count), sizeof(int), 1, OriginFile);
        if(FileByteInfo[i].count) types++;
    }
    return OK;
}

Status HuffmanDecoding(HuffmanTree T, int types, int FileSize, FILE
*OriginFile, FILE *NewFile){
    //按Huffman编码还原源文件
    unsigned char ReadBuff[1024];

```

```

int ReadSize;
int SizeCount = 0;
int i;
int pt = 2*types - 1; //从根开始
unsigned char ch = 0;
int BitCount = 0;
//每次读取1KB数据
while(1){
    ReadSize = fread(ReadBuff,1,1024,OriginFile);
    if(ReadSize == 0){
        //读取完毕
        break;
    }
    for(i=0;i<ReadSize;i++){
        ch = ReadBuff[i];
        for(BitCount = 0;BitCount < 8;BitCount++){
            if(ch & 0x80){
                //读取位为1
                pt = T[pt].rchild;
            }
            else{
                //读取位为0
                pt = T[pt].lchild;
            }
            if(T[pt].lchild == 0 && T[pt].rchild == 0){
                //找到叶子结点，输出对应字节
                fputc(T[pt].data,NewFile);
                pt = 2*types - 1;
                //pt重置为根
                SizeCount++;
                if(SizeCount == FileSize){
                    break;
                }
            }
            ch = ch << 1;
        }
    }
}
return OK;
}

Status HuffmanDecompress(Info *FileByteInfo){
    //解压过程
    char FileName[100] = {0};
    char FileType[FILE_TYPE_LENGTH] = {0};
    int types = 0,FileSize = 0;
    HuffmanTree T = NULL;
    FILE *OriginFile,*NewFile;
    printf("请输入文件路径:\n");
    scanf("%s",FileName);
    if(!(OriginFile = fopen(FileName,"rb"))){
        printf("打开文件错误! \n");
        exit(ERROR);
    }
    //打开压缩文件
    fread(FileType,sizeof(char),FILE_TYPE_LENGTH,OriginFile);
    //读入源文件类型
    ReNaming(FileName,FileType,2);

```

```

    if(!(NewFile = fopen(FileName, "wb"))){
        printf("打开文件错误! \n");
        exit(ERROR);
    }
    //创建源文件
    fread(&FileSize, sizeof(int), 1, OriginFile);
    //读入源文件大小
    GetInfo(OriginFile, FileByteInfo, types);
    //读入字节数信息
    InitHuffmanTree(T, FileByteInfo, types);
    //创建Huffman树
    HuffmanDecoding(T, types, FileSize, OriginFile, NewFile);
    //按Huffman编码还原源文件
    printf("解压完成! 文件存储路径为: %s\n", FileName);
}

Status main(){
    Info FileByteInfo[256]; //数组下标即为对应字节内容
    int i;
    HuffmanTree T = NULL;
    for(i = 0; i < 256; i++){
        FileByteInfo[i].count = 0;
        FileByteInfo[i].HCode = NULL;
    } //初始化
    while(1){
        switch(menu()){
            case 1:
                HuffmanCompress(FileByteInfo);
                break;
            case 2:
                HuffmanDecompress(FileByteInfo);
                break;
            case 0:
                printf("感谢您的使用, 再见! \n");
                return OK;
        }
        for(i = 0; i < 256; i++){
            FileByteInfo[i].count = 0;
            if(FileByteInfo[i].HCode) free(FileByteInfo[i].HCode);
            FileByteInfo[i].HCode = NULL;
        }
        if(T){
            free(T);
            T = NULL;
        } //清空数据
    }
}

```



图的遍历

```

#include<stdio.h>
#include<stdlib.h>
#define TRUE 1
#define FALSE 0

```

```

#define ERROR -1
#define OVERFLOW -2
#define OK 1
typedef int Status;

typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;

typedef struct QNode{
    int data;
    struct QNode *next;
}QNode;

typedef struct Queue{
    QNode *front;
    QNode *rear;
}LinkQueue;

Status SetArcs(MGraph &G){
    //输入各边
    int i,j,k;
    for(k = 0;k <G.arcnum;k++){
        scanf("%d %d",&i,&j);
        G.arcs[i][j] = 1;
        G.arcs[j][i] = 1;
    }
    return OK;
}

Status EnQueue(LinkQueue &Q,int data){
    QNode *p = (QNode*)malloc(sizeof(QNode));
    p->data = data;
    p->next = NULL;
    Q.rear->next = p;
    Q.rear = p;
    return OK;
}

Status DeQueue(LinkQueue &Q,int &data){
    QNode *p = Q.front->next;
    data = p->data;
    Q.front->next = p->next;
    free(p);
    if(!Q.front->next)
        Q.rear = Q.front;
    return OK;
}

Status QueueEmpty(LinkQueue Q){
    return (Q.front == Q.rear);
}

Status DFSTraverse(MGraph G,int start){
    //深度优先搜索
    void DFS(MGraph G,int u,int *visited);
    int *visited = (int*)malloc(sizeof(int)*(G.vexnum+1));

```

```

    if(!visited){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    }
    int i;
    for(i = 0;i <= G.vexnum;i++){
        visited[i] = FALSE;
        //初始化访问标记数组
    }
    DFS(G,start,visited);
    for(i = 1;i <= G.vexnum;i++){
        if(!visited[i]) DFS(G,i,visited);
    }
    putchar('\n');
    return OK;
}

void DFS(MGraph G,int u,int *visited){
    //深度优先搜索的递归算法
    visited[u] = TRUE;
    printf("%d ",u);
    int w;
    for(w = 1;w <= G.vexnum;w++){
        if(!visited[w] && G.arcs[u][w])
            DFS(G,w,visited);
    }
}

Status BFSTraverse(MGraph G,int start){
    void BFS(MGraph G,int u,int *visited,LinkQueue &Q);
    int *visited = (int*)malloc(sizeof(int)*(G.vexnum+1));
    if(!visited){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    }
    int i;
    for(i = 0;i <= G.vexnum;i++){
        visited[i] = FALSE;
        //初始化访问标记数组
    }
    LinkQueue Q;
    Q.front = (QNode*)malloc(sizeof(QNode));
    if(!Q.front){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    }
    Q.rear = Q.front;
    BFS(G,start,visited,Q);
    for(i = 1;i <= G.vexnum;i++){
        if(!visited[i]) BFS(G,i,visited,Q);
    }
    return OK;
}

void BFS(MGraph G,int u,int *visited,LinkQueue &Q){
    int w;
    visited[u] = TRUE;
    printf("%d ",u);
    EnQueue(Q,u);
    while(!QueueEmpty(Q)){

```

```

        DeQueue(Q,u);
        for(w = 1;w <= G.vexnum;w++){
            if(!visited[w] && G.arcs[u][w]){
                visited[w] = TRUE;
                printf("%d ",w);
                EnQueue(Q,w);
            }
        }
    }
}

int main(){
    MGraph G;
    int i,j,start;
    scanf("%d %d",&G.vexnum,&G.arcnum);
    G.arcs = (int**)malloc(sizeof(int)*(G.vexnum+1));
    if(!G.arcs){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    }
    for(i = 0;i <= G.vexnum;i++){
        G.arcs[i] = (int*)malloc(sizeof(int)*(G.vexnum+1));
        if(!G.arcs[i]){
            perror("内存分配失败\n");
            exit(OVERFLOW);
        }
        for(j = 0;j <= G.vexnum;j++){
            G.arcs[i][j] = 0;
        }
        //完成图邻接矩阵的创建与初始化
        SetArcs(G);
        //输入边的信息
        scanf("%d",&start);
        if(start >= 1 && start <= G.vexnum){
            DFSTraverse(G,start);
            BFSTraverse(G,start);
        }
        else printf("输入不合法\n");
        return OK;
    }
}

```

求通讯网的最小代价生成树

```

#include<stdio.h>
#include<stdlib.h>
#define TRUE 1
#define FALSE 0
#define ERROR -1
#define OVERFLOW -2
#define OK 1
#define INFINITY 2147483647
typedef int Status;

typedef struct MGraph{
    int **arcs;
    int vexnum,arcnum;
}MGraph;

```



```

typedef struct closedge{
    int adjvex;
    int lowcost;
}closedge;

Status SetArcs(MGraph &G){
    //输入各边
    int i,j,k;
    int right;
    for(k = 0;k < G.arcnum;k++){
        scanf("%d %d %d",&i,&j,&right);
        G.arcs[i][j] = right;
        G.arcs[j][i] = right;
    }
    return OK;
}

int MiniSpanTree_PRIM(MGraph G){
    //从顶点1出发用Prim算法构建最小生成树
    //返回树各边的长度之和
    closedge *close = (closedge*)malloc(sizeof(closedge)*(G.vexnum+1));
    if(!close){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    }
    int i,j,k;
    int rightsum = 0;
    int min;
    for(i = 2;i <= G.vexnum;i++){
        close[i].adjvex = 1;
        close[i].lowcost = G.arcs[1][i];
    } //初始化close数组
    close[1].lowcost = 0;
    for(i = 1;i < G.vexnum;i++){
        min = INFINITY;
        for(j = 1;j <= G.vexnum;j++){
            if(close[j].lowcost && close[j].lowcost < min){
                min = close[j].lowcost;
                k = j;
            }
        }
        //找到距离最短的下个顶点k
        rightsum += G.arcs[close[k].adjvex][k];
        //增加总长
        close[k].lowcost = 0;
        //第k顶点并入已有树
        for(j = 1;j <= G.vexnum;j++){
            if(G.arcs[k][j] < close[j].lowcost){
                close[j].adjvex = k;
                close[j].lowcost = G.arcs[k][j];
            }
        } //更新close数组
    }
    return rightsum;
}

int MiniSpanTree_KRUS(MGraph G){

```

```

//从顶点1出发用Kruskal算法构建最小生成树
//返回树各边的长度之和
int *vexgroup = (int*)malloc(sizeof(int)*(G.vexnum+1));
if(!vexgroup){
    perror("内存分配失败\n");
    exit(OVERFLOW);
} //创建顶点分组标记数组
int i,j,k;
for(i = 1;i <= G.vexnum;i++){
    vexgroup[i] = i;
} //初始化顶点分组标记
int rightsum = 0;
int min,mi,mj;
for(k = 1;k < G.vexnum;k++){
    min = INFINITY;
    for(i = 1;i <= G.vexnum;i++){
        for(j = i+1;j <= G.vexnum;j++){
            if(vexgroup[i] != vexgroup[j] && min > G.arcs[i][j]){
                min = G.arcs[i][j];
                mi = i;mj = j;
            }
        }
    }
    //找到处于不同连通分量的最短边mi-mj
    rightsum += min;
    //增加总长
    for(i = 1;i <= G.vexnum;i++){
        if(vexgroup[i] == vexgroup[mj])
            vexgroup[i] = vexgroup[mi];
    }
    //更新顶点分组标记
}
return rightsum;
}

int main(){
    MGraph G;
    int i,j;
    scanf("%d %d",&G.vexnum,&G.arcnum);
    G.arcs = (int**)malloc(sizeof(int)*(G.vexnum+1));
    if(!G.arcs){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    }
    for(i = 0;i <= G.vexnum;i++){
        G.arcs[i] = (int*)malloc(sizeof(int)*(G.vexnum+1));
        if(!G.arcs[i]){
            perror("内存分配失败\n");
            exit(OVERFLOW);
        }
        for(j = 0;j <= G.vexnum;j++)
            G.arcs[i][j] = INFINITY;
    }
    //完成图邻接矩阵的创建与初始化
    SetArcs(G);
    //输入边的信息
    printf("Prim算法结果为: %d\n",MiniSpanTree_PRIM(G));
    printf("Kruskal算法结果为: %d\n",MiniSpanTree_KRUS(G));
}

```

```

        //输出最小生成树边长度之和
        return OK;
    }

```

铁路交通网的最短路径

```

#include<stdio.h>
#include<stdlib.h>
#define TRUE 1
#define FALSE 0
#define ERROR -1
#define OVERFLOW -2
#define OK 1
#define INFINITY 2147483647
typedef int Status;

typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;

typedef struct closedge{
    int adjvex;
    int lowcost;
}closedge;

Status SetArcs(MGraph &G){
    //输入各边
    int i, j, k;
    int right;
    for(k = 0; k < G.arcnum; k++){
        scanf("%d %d %d", &i, &j, &right);
        G.arcs[i][j] = right;
        G.arcs[j][i] = right;
    }
    return OK;
}

int ShortestPath_DIJ(MGraph G, int start, int end){
    //用Dijkstra算法计算start到end的最短路径
    //返回最短路径的长度
    int *distance = (int*)malloc(sizeof(int)*(G.vexnum+1));
    if(!distance){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    } //存储到顶点的最短距离
    int *final = (int*)malloc(sizeof(int)*(G.vexnum+1));
    if(!final){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    } //存储是否经过顶点
    int i, j, k;
    int min;
    for(i = 1; i <= G.vexnum; i++){
        distance[i] = G.arcs[start][i];
        final[i] = FALSE;
    } //初始化

```

```

final[start] = TRUE;
distance[start] = 0;
for(i = 1; i < G.vexnum; i++){
    min = INFINITY;
    for(j = 1; j <= G.vexnum; j++){
        if(!final[j] && min > distance[j]){
            k = j;
            min = distance[j];
        }
    }
    //找到未经过的最近顶点k
    final[k] = TRUE;
    //标记经过k
    for(j = 1; j <= G.vexnum; j++){
        if(!final[j] && distance[j] > min+G.arcs[k][j]){
            distance[j] = min+G.arcs[k][j];
        }
    }
    //更新未经过顶点最近距离
}
//完成各顶点最近距离计算
return distance[end];
}

int main(){
    MGraph G;
    int i, j;
    int start, end;
    scanf("%d %d", &G.vexnum, &G.arcnum);
    G.arcs = (int**)malloc(sizeof(int)*(G.vexnum+1));
    if(!G.arcs){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    }
    for(i = 0; i <= G.vexnum; i++){
        G.arcs[i] = (int*)malloc(sizeof(int)*(G.vexnum+1));
        if(!G.arcs[i]){
            perror("内存分配失败\n");
            exit(OVERFLOW);
        }
        for(j = 0; j <= G.vexnum; j++)
            G.arcs[i][j] = INFINITY;
    }
    //完成图邻接矩阵的创建与初始化
    SetArcs(G);
    //输入边的信息
    scanf("%d %d", &start, &end);
    printf("%d", ShortestPath_DIJ(G, start, end));
    return OK;
}

```

显示图

```

#include<stdio.h>
#include<graphics.h>
#include<conio.h>
#include<stdlib.h>

```

```

#include<math.h>
#define TRUE 1
#define FALSE 0
#define OK 1
#define T_0 0.00001
#define X_0 100
#define K_1 0.5
#define K_2 0.00001
typedef int Status;

typedef struct MGraph{
    int **arcs;
    int vexnum, arcnum;
}MGraph;

typedef struct Vector{
    double x;
    double y;
}Vector;

typedef struct Vertex{
    double x;
    double y;
    Vector v;
    Vector a;
}Vertex;

Status SetArcs(MGraph &G){
    //输入各边
    int i, j, k;
    for(k = 0; k < G.arcnum; k++){
        scanf("%d %d", &i, &j);
        getchar();
        G.arcs[i][j] = 1;
        G.arcs[j][i] = 1;
    }
    return OK;
}

void GetA(MGraph G, Vertex *ver){
    //计算各顶点的加速度
    int i, j;
    for(i = 1; i <= G.vexnum; i++){
        ver[i].a.x = 0;
        ver[i].a.y = 0;
        for(j = 1; j <= G.vexnum; j++){
            if(j != i){
                if(ver[j].x > ver[i].x){
                    if(G.arcs[i][j] == 1)
                        ver[i].a.x += K_1*(ver[j].x-ver[i].x-X_0); //计算引力
                    ver[i].a.x -= K_2/(ver[j].x-ver[i].x)/(ver[j].x-ver[i].x);
                }
            }
            else{
                if(G.arcs[i][j] == 1)
                    ver[i].a.x -= K_1*(ver[i].x-ver[j].x-X_0); //计算引力
                    ver[i].a.x += K_2/(ver[i].x-ver[j].x)/(ver[i].x-ver[j].x);
            }
        }
    }
    //计算斥力
}

```

```

        }
        if(ver[j].y > ver[i].y){
            if(G.arcs[i][j] == 1)
                ver[i].a.y += K_1*(ver[j].y-ver[i].y-x_0); //计算引力
            ver[i].a.y -= K_2/(ver[j].y-ver[i].y)/(ver[j].y-ver[i].y);
//计算斥力
        }
        else{
            if(G.arcs[i][j] == 1)
                ver[i].a.y -= K_1*(ver[i].y-ver[j].y-x_0); //计算引力
            ver[i].a.y += K_2/(ver[i].y-ver[j].y)/(ver[i].y-ver[j].y);
//计算斥力
        }
    }
}

void GetNextStatus(MGraph G,Vertex *ver){
//计算下一时刻的速度与坐标
int i;
for(i = 1;i <= G.vexnum;i++){
    ver[i].x += ver[i].v.x * T_0 + ver[i].a.x * T_0 * T_0;
    ver[i].v.x += ver[i].a.x * T_0;
    ver[i].y += ver[i].v.y * T_0 + ver[i].a.y * T_0 * T_0;
    ver[i].v.y += ver[i].a.y * T_0;
}
}

void PrintGraph(MGraph G,Vertex *ver){
//输出图
initgraph(1440,720);
setorigin(720,360);
setbkcolor(WHITE);
cleardevice();
setfillcolor(WHITE);
setlinecolor(BLACK);
settextcolor(BLACK);
//完成初始化
int i,j;
char s[5];
for(i = 1;i <= G.vexnum;i++){
    for(j = i;j <= G.vexnum;j++){
        if(G.arcs[i][j]){
            line(ver[i].x,ver[i].y,ver[j].x,ver[j].y);
        }
    }
} //画边
for(i = 1;i <= G.vexnum;i++){
    fillcircle(ver[i].x,ver[i].y,16);
    sprintf(s,"%d",i);
    outtextxy(ver[i].x-3,ver[i].y-7,s);
} //画圆
getchar();
}

int main(){
    MGraph G;

```

```

int i,j;
scanf("%d %d",&G.vexnum,&G.arcnum);
G.arcs = (int**)malloc(sizeof(int)*(G.vexnum+1));
if(!G.arcs){
    perror("内存分配失败\n");
    exit(OVERFLOW);
}
for(i = 0;i <= G.vexnum;i++){
    G.arcs[i] = (int*)malloc(sizeof(int)*(G.vexnum+1));
    if(!G.arcs[i]){
        perror("内存分配失败\n");
        exit(OVERFLOW);
    }
    for(j = 0;j <= G.vexnum;j++){
        G.arcs[i][j] = 0;
    }
}
//完成图邻接矩阵的创建与初始化
SetArcs(G);
//输入边的信息
Vertex *ver = (Vertex*)malloc(sizeof(Vertex)*(G.vexnum+1));
for(i = 1;i <= G.vexnum;i++){
    ver[i].a.x = 0;
    ver[i].a.y = 0;
    ver[i].v.x = 0;
    ver[i].v.y = 0;
    ver[i].x = 250 * cos((i-1) * 2 * 3.1415926 / G.vexnum);
    ver[i].y = 250 * sin((i-1) * 2 * 3.1415926 / G.vexnum);
}
//建立各顶点并初始化
for(i = 0;i <= 1000000;i++){
    GetA(G,ver);
    GetNextStatus(G,ver);
}
for(i = 1;i <= G.vexnum;i++){
    printf("%d %lf %lf\n",i,ver[i].x,ver[i].y);
}
PrintGraph(G,ver);
return OK;
}

```

哈希表

```

#include<stdio.h>
#include<stdlib.h>
#define OK 1
#define ERROR -1
#define SUCCESS 1
#define UNSUCCESS 0
#define DUPLICATE -1
#define NULLKEY 2147483647
typedef int Status;

typedef struct CNode{
    int elem;
    struct CNode *next;
}CNode,*Chain;

```

```

typedef struct CHTable{
    Chain *ChainHash;
    int *ChainLength;
    int count;
}CHashTable;

typedef struct HTable{
    int *elem;
    int *successtime;
    int *failtime;
    int count;
}HashTable;

int tablesize,num;

int Hash(int x){
    //计算哈希地址
    return x % tablesize;
}

Status InitTable(HashTable &HT,CHashTable &CHT,int *data){
    int i,k,count = 0;
    int address;
    CNode *p = NULL,*q = NULL;
    HT.elem = (int*)malloc(sizeof(int)*tablesize);
    HT.successtime = (int*)malloc(sizeof(int)*tablesize);
    HT.failtime = (int*)malloc(sizeof(int)*tablesize);
    HT.count = 0;
    CHT.ChainHash = (Chain*)malloc(sizeof(Chain)*tablesize);
    CHT.ChainLength = (int*)malloc(sizeof(int)*tablesize);
    CHT.count = 0;
    for(int i = 0;i < tablesize;i++){
        HT.elem[i] = NULLKEY;
        HT.successtime[i] = 0;
        HT.failtime[i] = 0;
        CHT.ChainHash[i] = NULL;
        CHT.ChainLength[i] = 0;
    }
    //创建与初始化
    for(i = 0;i < num;i++){
        count = 1;
        address = Hash(data[i]);
        //插入HT表内容
        HT.count++;
        k = address;
        while(HT.elem[k] != NULLKEY){
            k = (k+1)%tablesize;
            count++;
        }
        HT.successtime[k] = count;
        //找到插入位置，记录成功查找次数
        HT.elem[k] = data[i];
        //完成赋值
        //插入CHT表内容
        CHT.count++;
        p = (CNode*)malloc(sizeof(CNode));
        p->elem = data[i];
    }
}

```



```

        p->next = NULL;
        //完成新结点创建
        q = CHT.ChainHash[address];
        if(!q)
            CHT.ChainHash[address] = p;
        else{
            while(q->next && q->next->elem < data[i])
                q = q->next;
            p->next = q->next;
            q->next = p;
        }
        //按顺序插入结点
        CHT.ChainLength[address]++;
    }
    for(i = 0; i < tablesize; i++){
        //计算失败查找次数
        count = 1;
        k = i;
        while(HT.elem[k] != NULLKEY){
            k = (k+1)%tablesize;
            count++;
        }
        HT.failtime[i] = count;
    }
    return OK;
}

```

```

Status PrintTable(HashTable HT, CHashTable CHT){
    int i, j;
    double TotalSTime, TotalFTime;
    CNode *p = NULL;
    printf("HT1内容为: \n");
    printf("表中的关键字: \n");
    for(i = 0; i < tablesize; i++){
        printf("%-5d", i);
        if(HT.elem[i] != NULLKEY)
            printf("%d", HT.elem[i]);
        else
            printf("-");
        putchar('\n');
    }
    TotalSTime = 0;
    printf("成功查找次数: \n");
    for(i = 0; i < tablesize; i++){
        printf("%-5d", i);
        if(HT.elem[i] != NULLKEY){
            printf("%d", HT.successtime[i]);
            TotalSTime += HT.successtime[i];
        }
        else
            printf("-");
        putchar('\n');
    }
    TotalFTime = 0;
    printf("失败查找次数: \n");
    for(i = 0; i < tablesize; i++){
        printf("%-5d", i);
        printf("%d", HT.failtime[i]);
    }
}

```

```

        TotalFTime += HT.failtime[i];
        putchar('\n');
    }
    printf("查找成功的平均查找长度: %.21f\n", TotalSTime/num);
    printf("查找失败的平均查找长度: %.21f\n\n", TotalFTime/tablesiz);
    //输出HT1内容
    printf("HT2内容为: \n");
    printf("表中的关键字: \n");
    for(i = 0; i < tablesiz; i++){
        printf("%-5d", i);
        if(CHT.ChainHash[i]){
            p = CHT.ChainHash[i];
            while(p){
                printf("%-6d", p->elem);
                p = p->next;
            }
        }
        else
            printf("-");
        putchar('\n');
    }
    TotalSTime = 0;
    printf("成功查找次数: \n");
    for(i = 0; i < tablesiz; i++){
        printf("%-5d", i);
        if(CHT.ChainHash[i]){
            p = CHT.ChainHash[i];
            j = 1;
            while(p){
                printf("%-6d", j);
                TotalSTime += j;
                j++;
                p = p->next;
            }
        }
        else
            printf("0");
        putchar('\n');
    }
    TotalFTime = 0;
    printf("失败查找次数: \n");
    for(i = 0; i < tablesiz; i++){
        printf("%-5d", i);
        printf("%-6d", CHT.ChainLength[i]+1);
        TotalFTime += CHT.ChainLength[i]+1;
        putchar('\n');
    }
    printf("查找成功的平均查找长度: %.21f\n", TotalSTime/num);
    printf("查找失败的平均查找长度: %.21f\n", TotalFTime/tablesiz);
    return OK;
}

int main(){
    FILE *fp = NULL;
    char filename[20] = {0};
    HashTable HT;
    CHashTable CHT;
    int *data = NULL;
    int i;

```

```
printf("请输入文件路径: ");
scanf("%s", filename);
putchar('\n');
if(!(fp = fopen(filename, "r"))){
    perror("文件打开失败! \n");
    exit(ERROR);
}
fscanf(fp, "%d", &num);
//读取记录个数
data = (int*)malloc(sizeof(int)*num);
for(i = 0; i < num; i++){
    fscanf(fp, "%d", &data[i]);
}
//读入数据
fscanf(fp, "%d", &tablesize);
//读入除留余数法的模(表长)
InitTable(HT, CHT, data);
PrintTable(HT, CHT);
return OK;
}
```